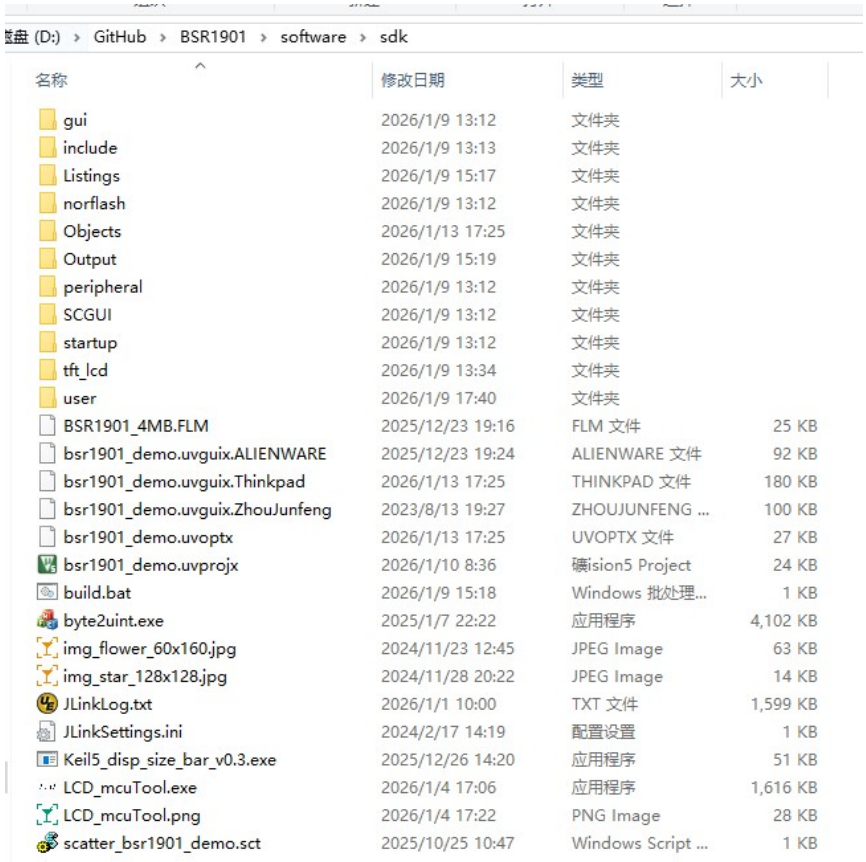
BSR1901 之 AI-IDE 人工智能开发环境

目录

1 KEIL 和 VS Code 开发环境 1
2 在 VS Code 中安装 Keil 助手和 Cline 5
3 接入 DeepSeek 8
4 Ollama 本地部署 DeepSeek-R1 大模型 11
5 LM Studio 本地部署 Gemma4 大模型 18
6 Claude + DeepSeek-V4 大模型 20
7 Cline+ NVIDIA 大模型 22
8 打印 log 信息 24
9 在线升级 26
10 量产 JFlash 烧录 27
11 修订记录 31

1 KEIL 和 VS Code 开发环境

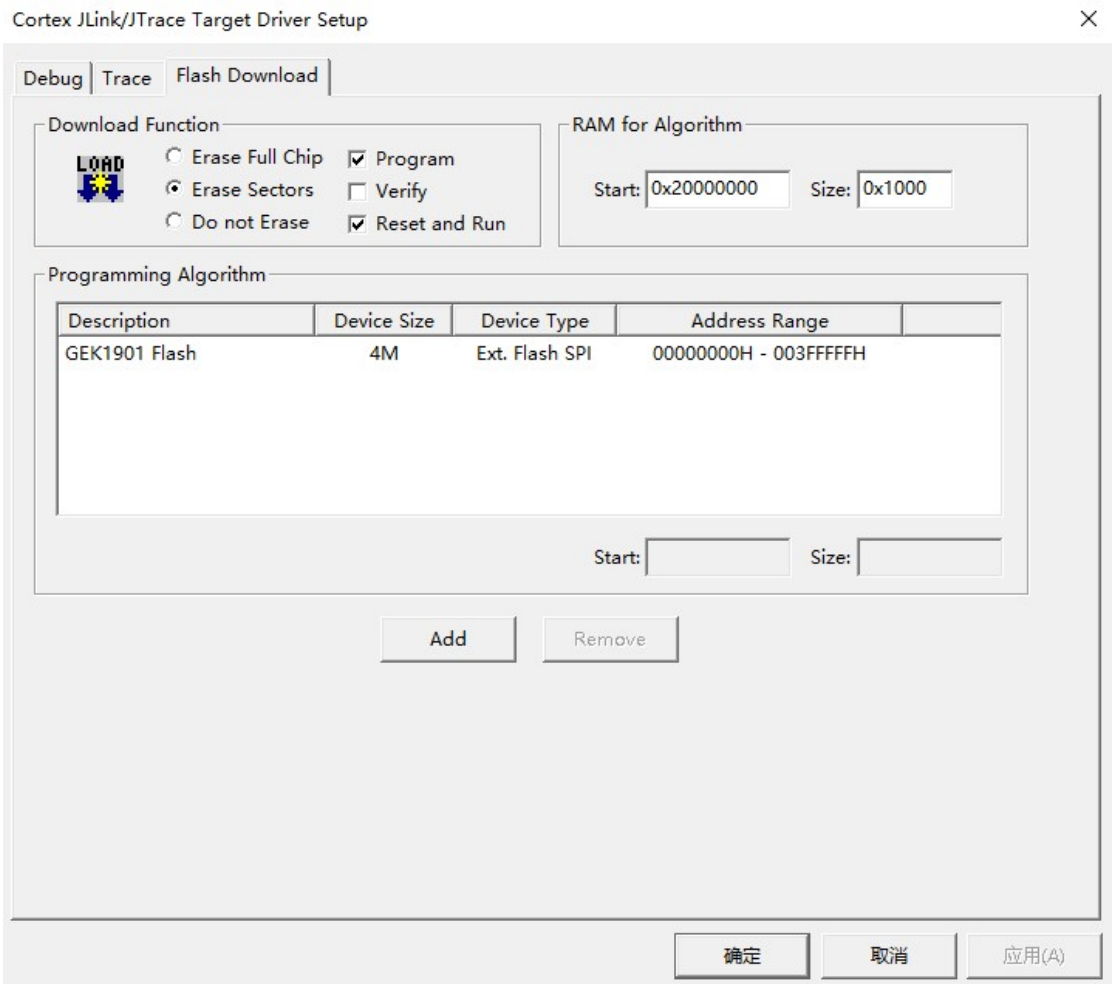
BSR 提供的 SDK 代码是基于 ARM Keil 或者 VS Code 开发的，代码放置在电脑的文件夹时，层级不要太深（不要超过三级），同时**文件夹命名时，不要出现中文字样**，否则可能导致 Keil 下载代码时异常。完整的 SDK 软件代码：



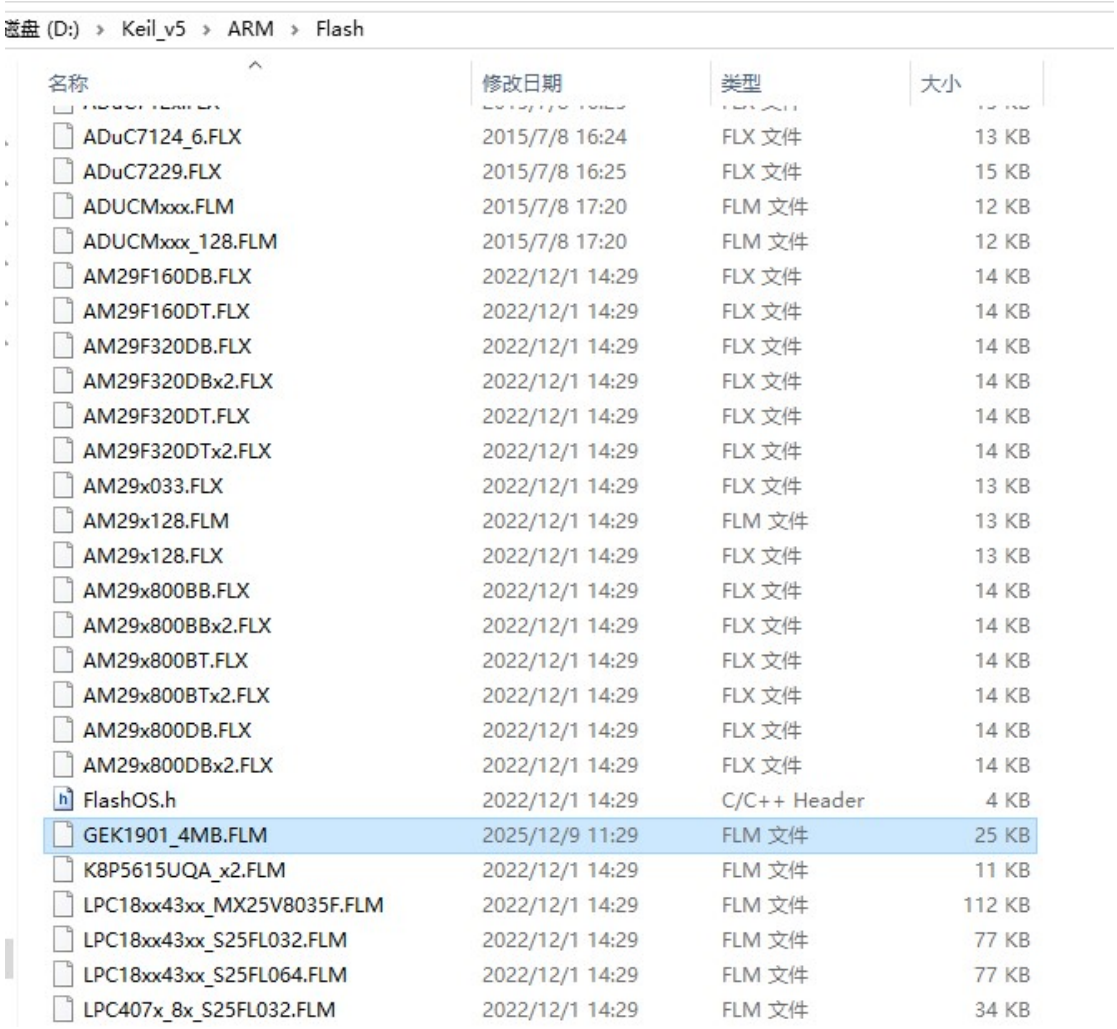
可以为客户提供的 Demo 板：



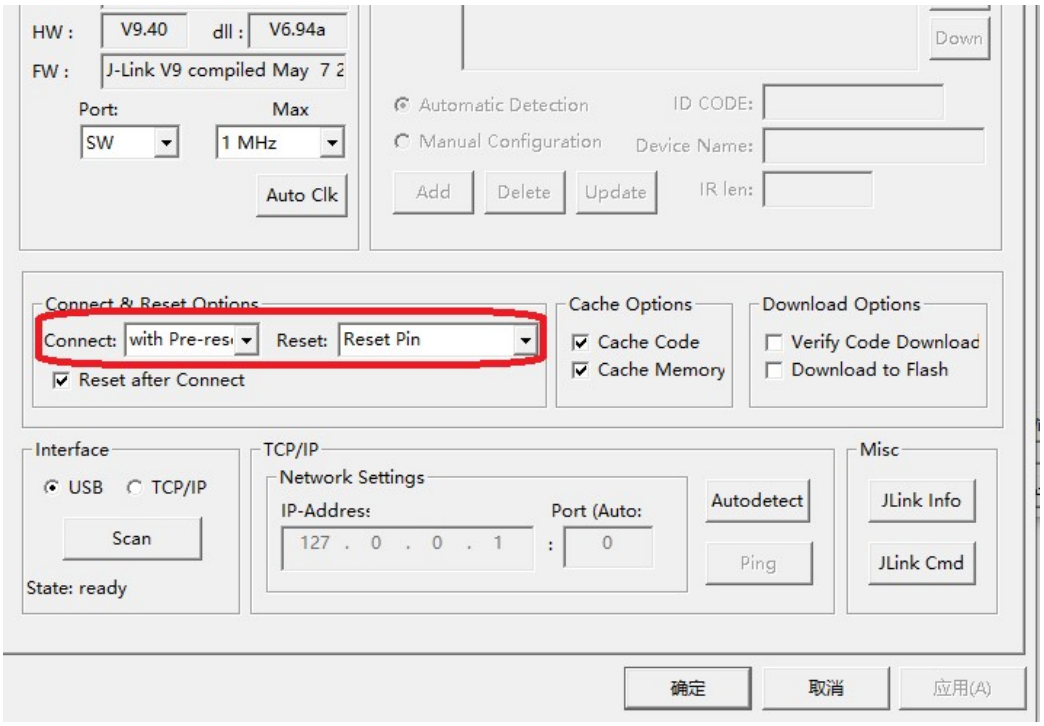
添加 Flash 下载算法

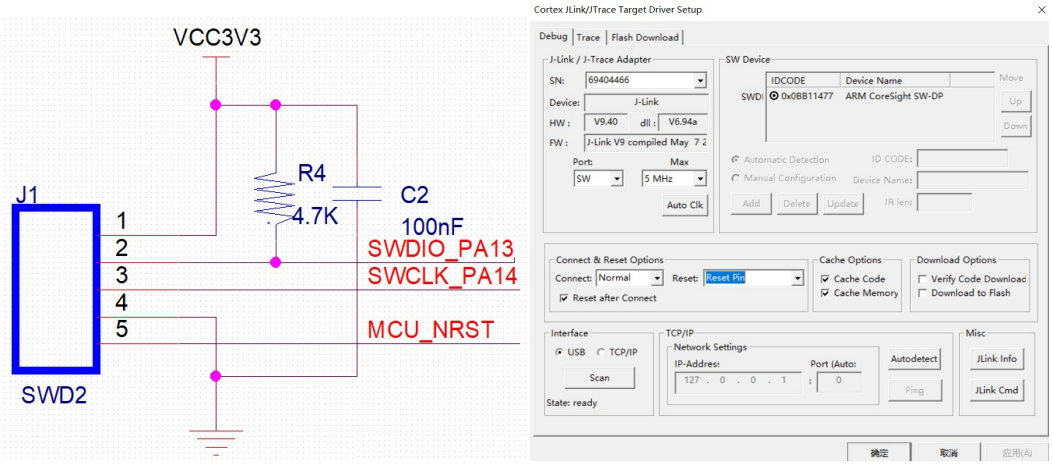


把 SDK 提供的 BSR1901.FLM 文件放到: C:\Keil_v5\ARM\Flash 目录下，如下图所示。

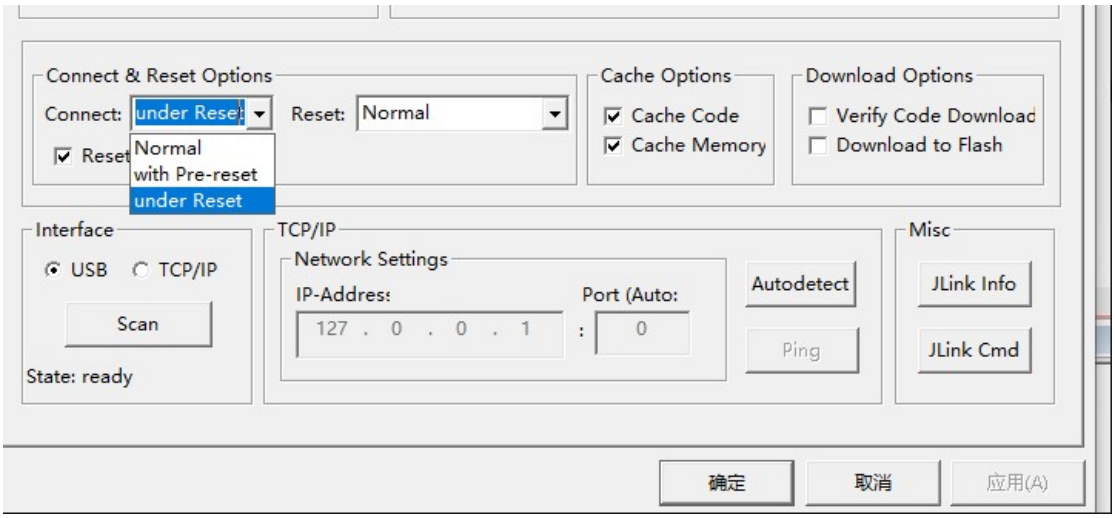


如果在硬件连接了 reset pin 的条件下，碰到异常的提示，可以设置一下红色框框：





特别说明一下，1901 芯片的 PAD01 和 PAD00 是 SWD 下载口，如果把 SWD 和 SWCLK 复用成 GPIO 口， 一旦程序运行起来了， SWD 和 SWCLK 就无法下载程序了。 这个时候需要把芯片的 RESETn 接到调试器对应的管脚上去，下载时先复位 CPU ，让 I/O 回归默认状态（SWD/SWCLK 功能），就能正常下载了。调试器在芯片刚复位、还没跑你代码的瞬间，抢先建立 SWD 连接。

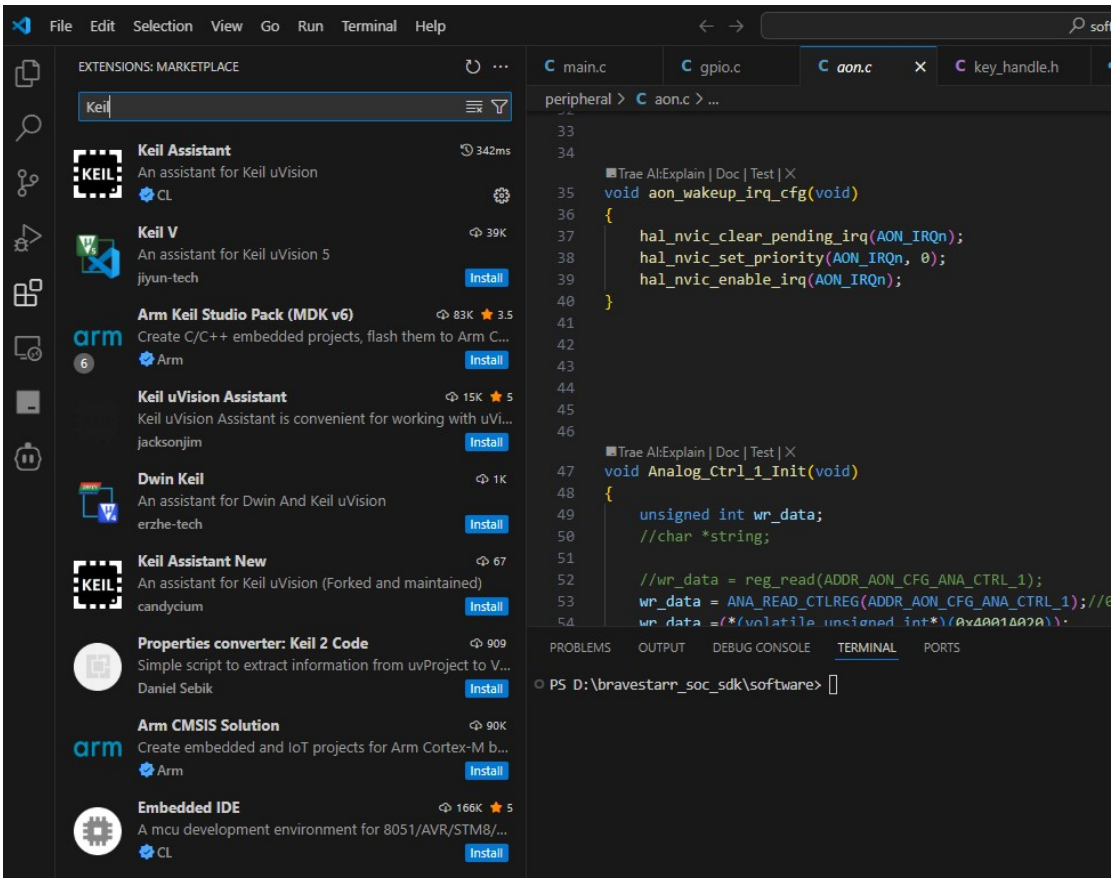


下载前勾选：硬件复位后连接 / Connect under reset

- Connect 选 under Reset ☑
- Reset 选 Normal ☑
- 勾选 Reset ☑

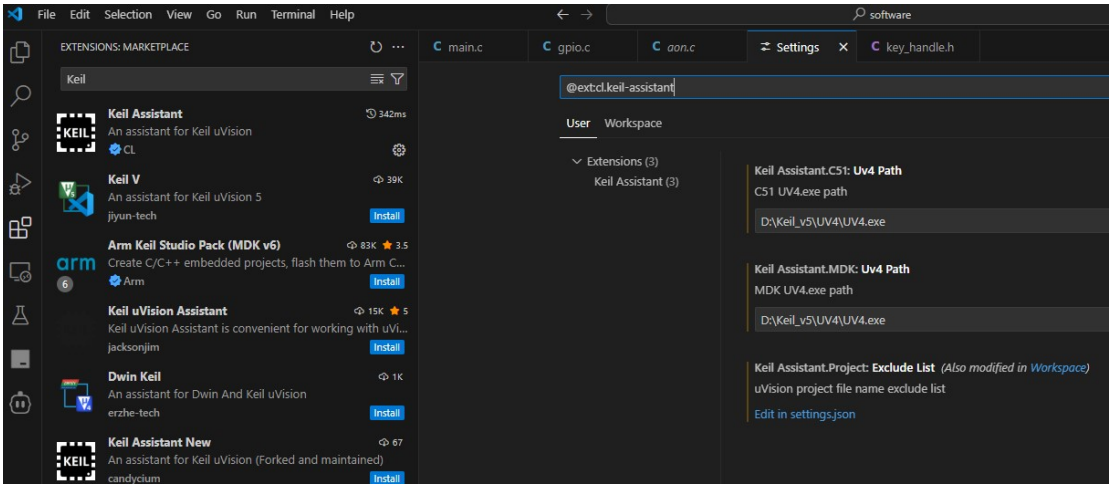
选项	适用场景
Normal（正常连接）	芯片空闲、SWD 口未被复用
with Pre-reset（预复位）	芯片运行中，需要先复位再连接
under Reset（复位下连接）	SWD 口被复用、芯片卡死、低功耗模式

2 在 VS Code 中安装 Keil 助手和 Cline



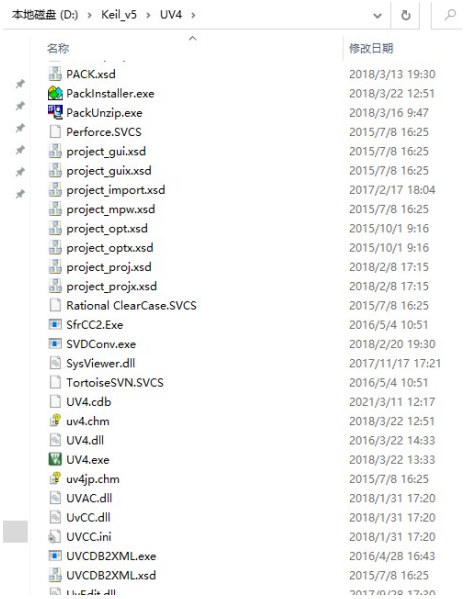
添加 Keil 环境变量 (让 VS Code 能够顺利找到我们的 Keil)

在 Keil 助手的 setting 菜单中，指定一下本地电脑的 Keil 工具的路径。

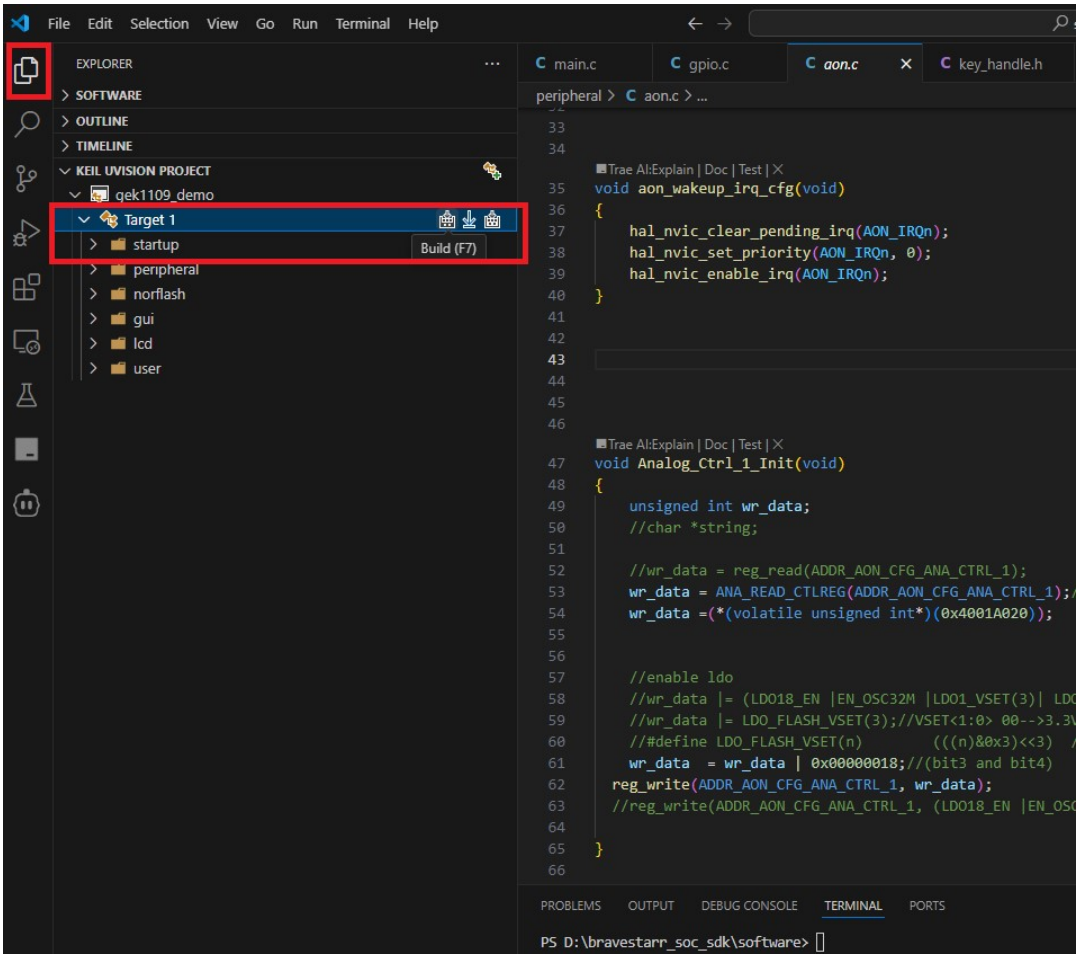


我的工程中没有 C51 的部分,可以不填。

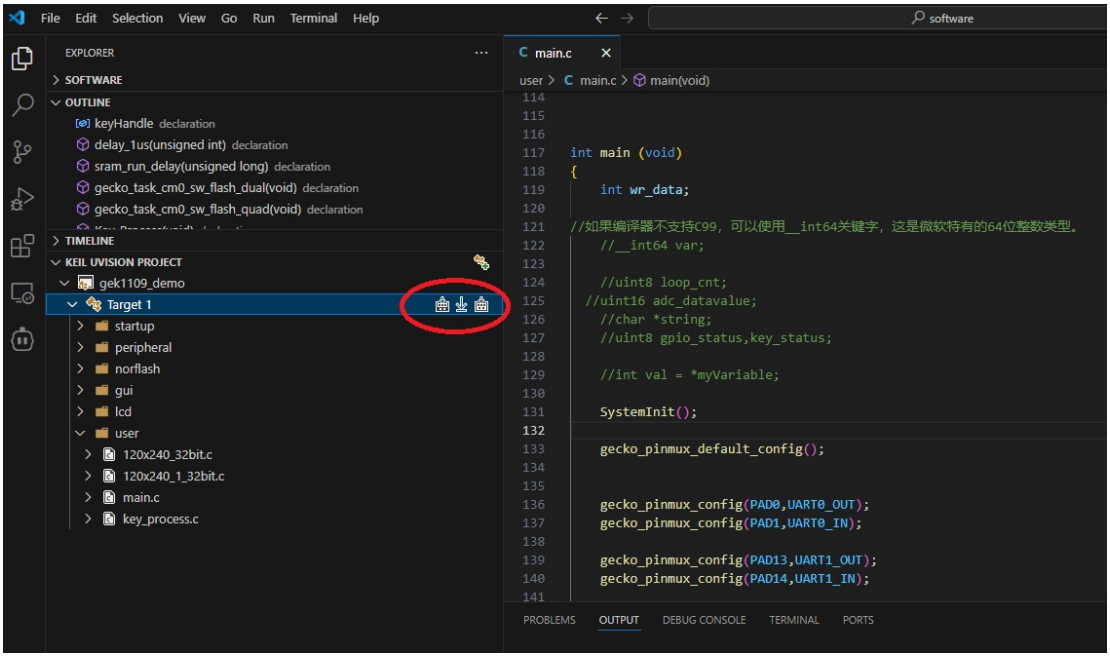
找到你本地电脑上安装的 Keil，把这个路径复制到 Keil Assistant.



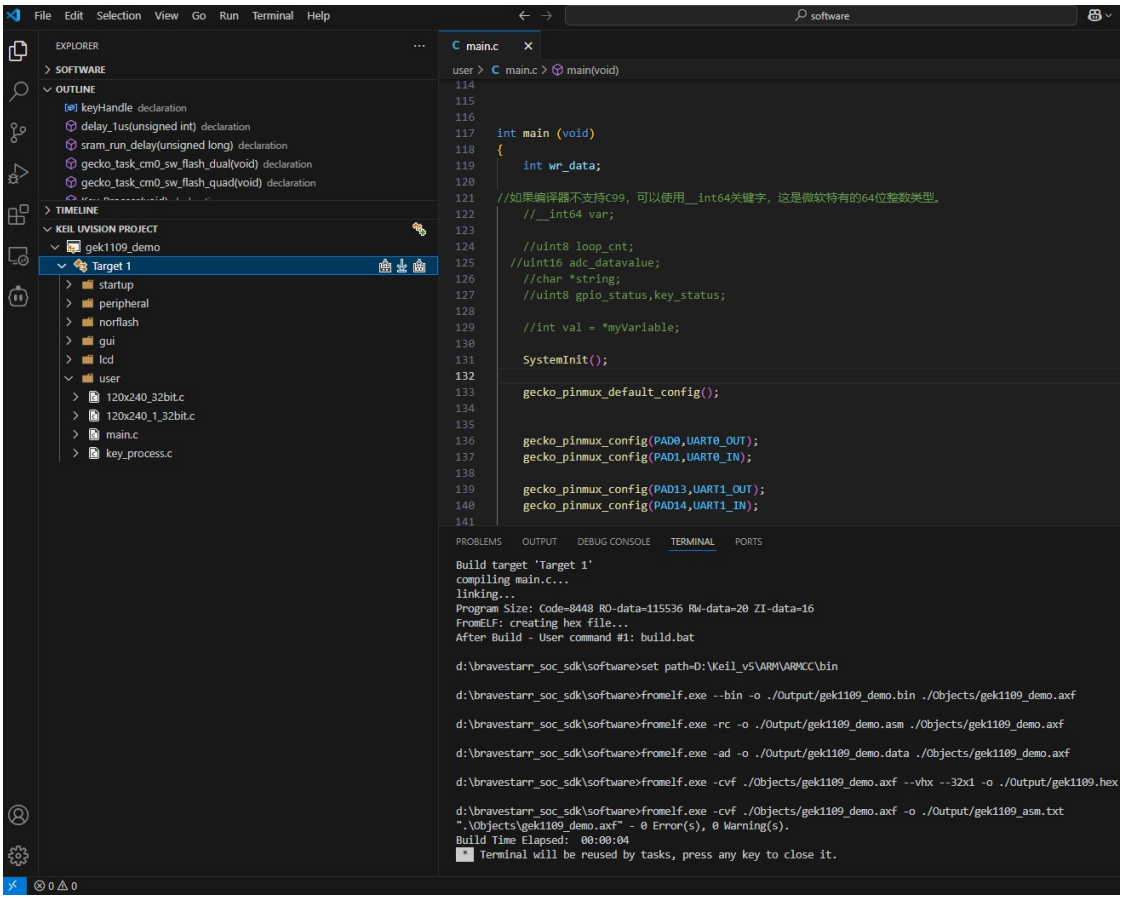
加载 Keil 工程文件



在 VS Code 中进行编译工程以及烧录程序

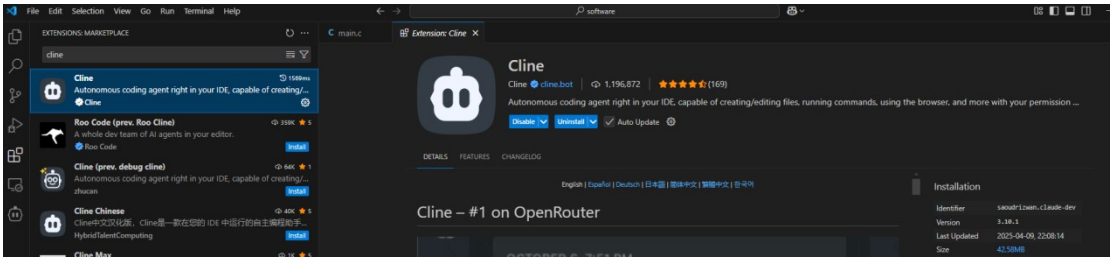


编译出来的结果跟 Keil 是一样的。



可以看到编译出来的信息是正确的。

在 VS Code 中安装 Cline

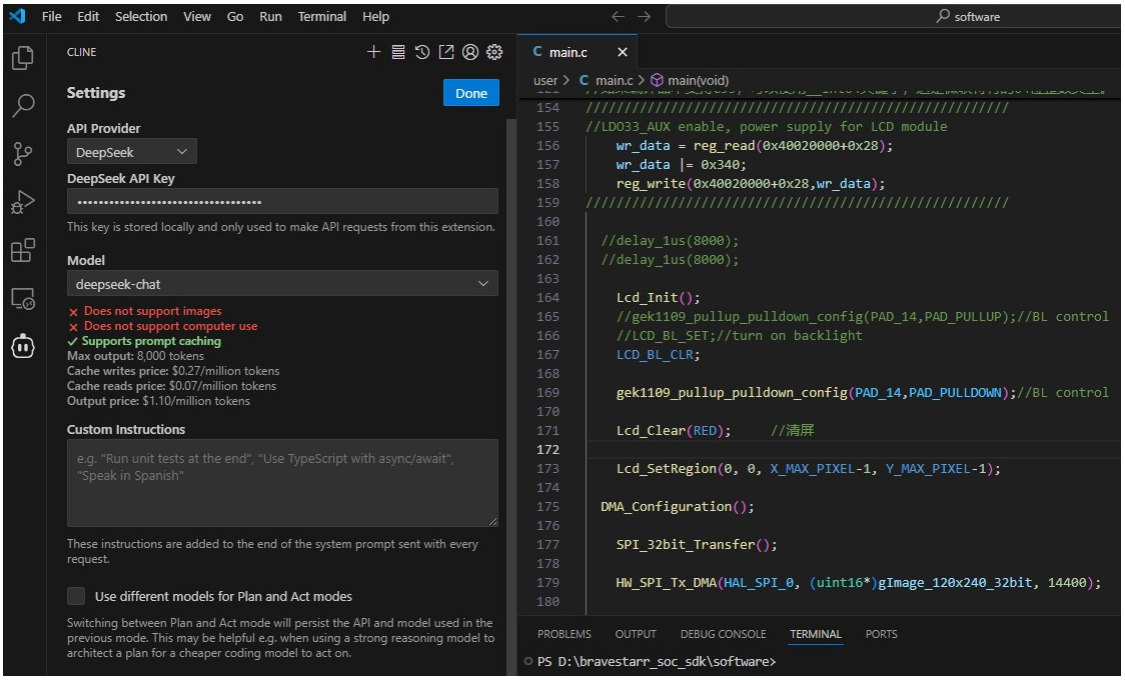


Cine 是 Visual Studio Code 中非常重要的一个插件，可以认为是一个 MCP 的 Client。

同时我们需要在 Cline 的 Setting 菜单中，添加你期望接入的大模型，在这里，我们选择 DeepSeek API 接入。

3 接入 DeepSeek

推荐直接使用 DeepSeek 的 API 接入



在申请 DeepSeek API，获取 API Key 的时候，这个过程是需要付费的，我已经付费试用过了。

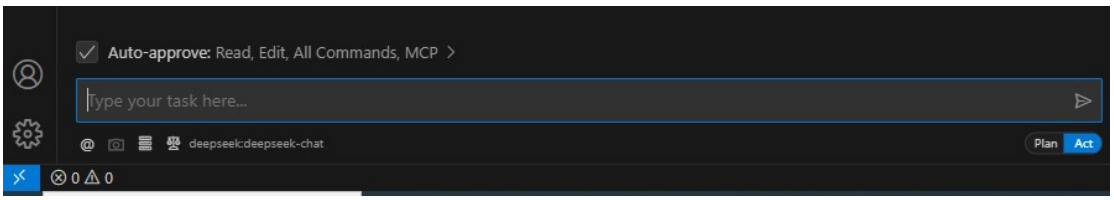


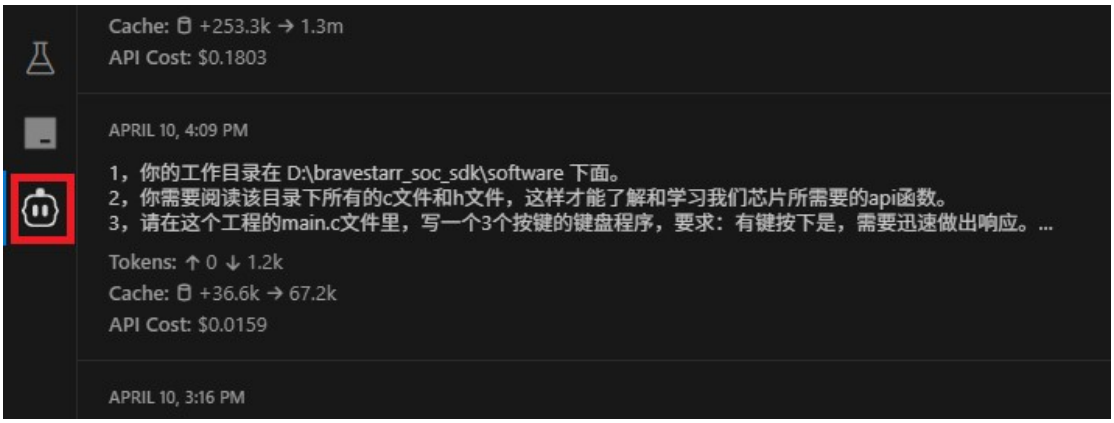
申请 API Keys 的地址：https://platform.deepseek.com/api_keys



Cline 结合 DeepSeek 写代码还是非常快，对代码的理解比人快。

下面我们让 Cline 来调用 DeepSeek 在我们的 1109 sdk 的基础，按照我们的要求，自动完成我们代码需求：

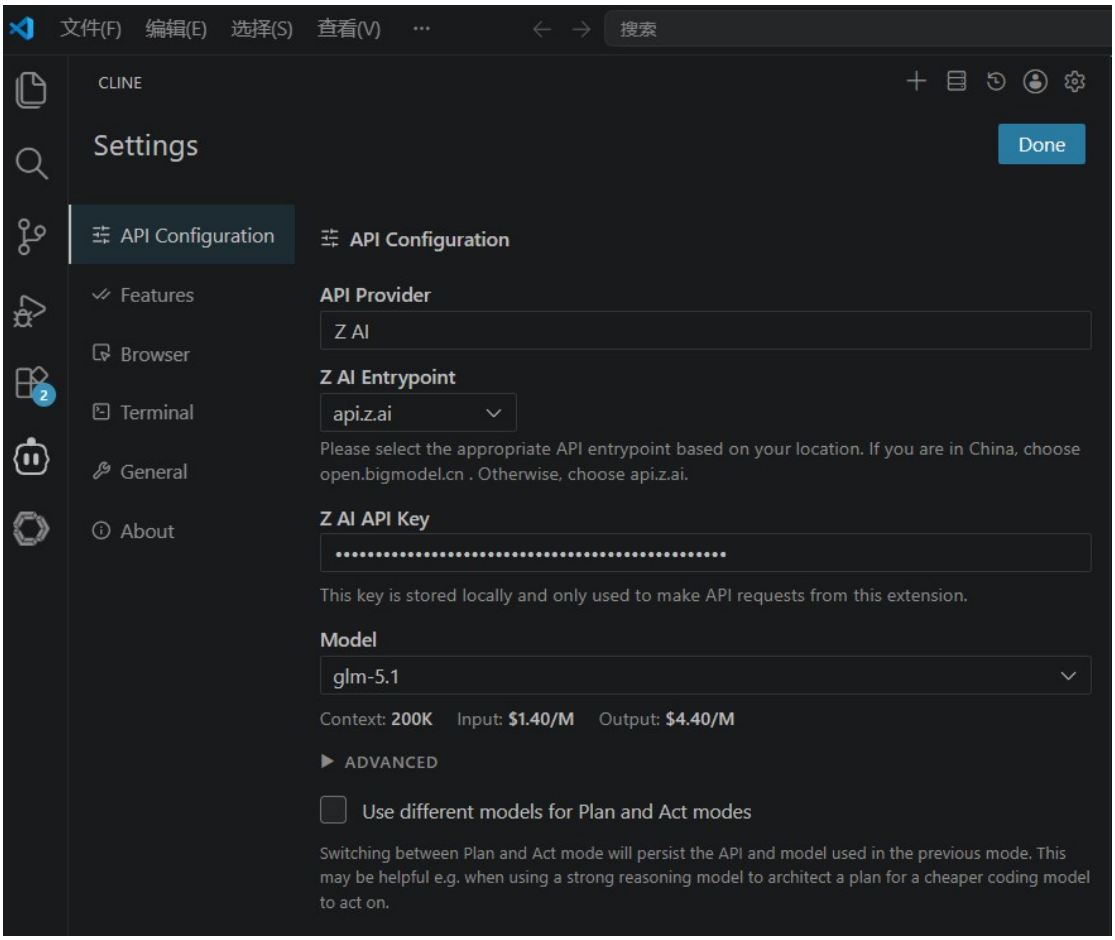




上面的这个提示词，是我给 Cline 下达新任务写的，工程师可以参考我这个提示词，构建新的任务提示词。

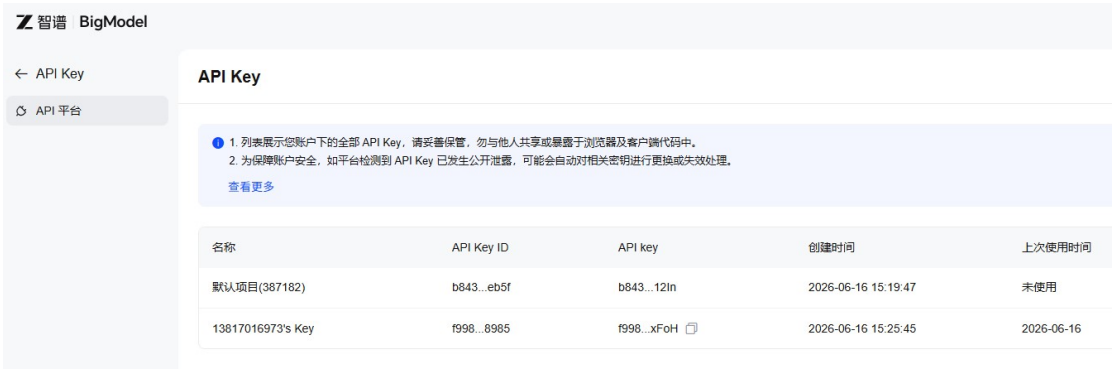
Cline 完成代码之后，可以在 Keil 中下载代码到 1901 芯片中，验证是否已经符合工程师的要求。

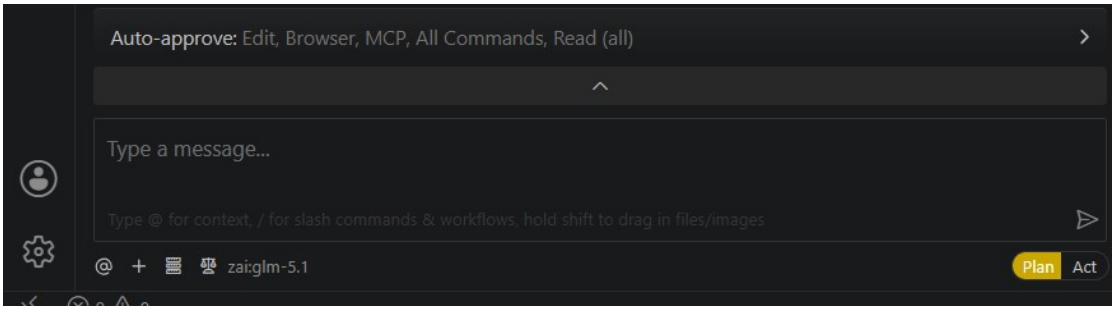
增加智谱的 glm-5.1 大模型:



https://bigmodel.cn/apikey/platform

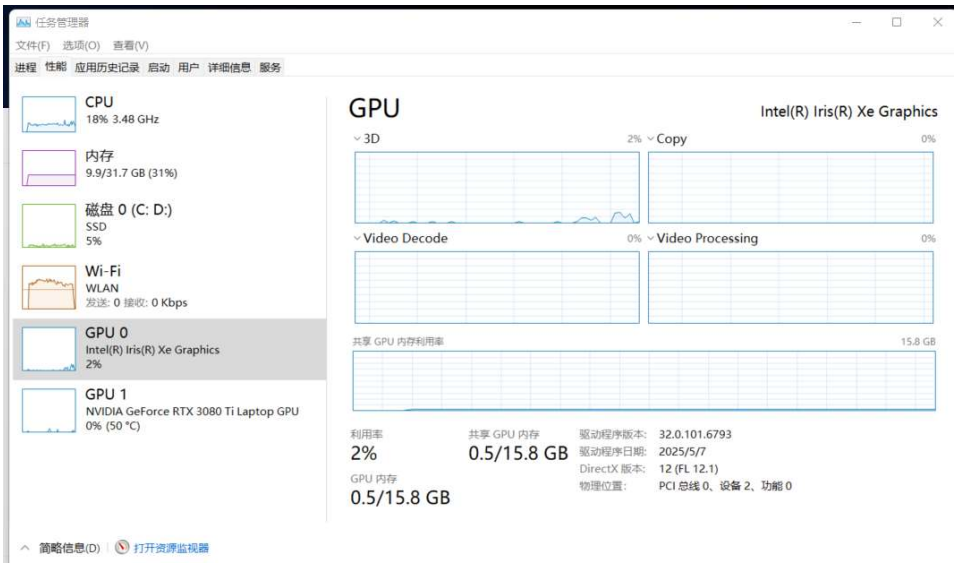
申请 api key

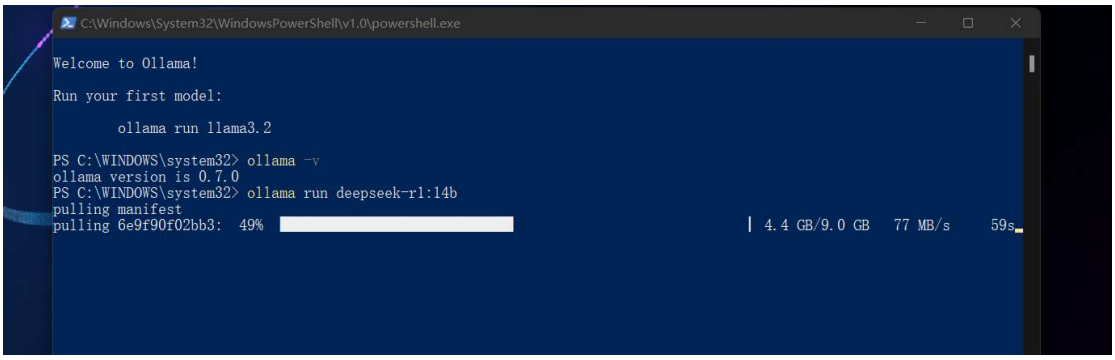




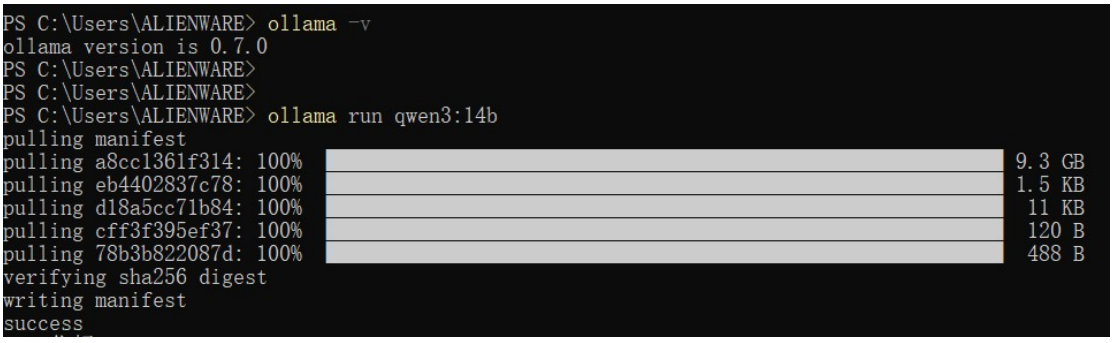
4 Ollama 本地部署 DeepSeek-R1 大模型

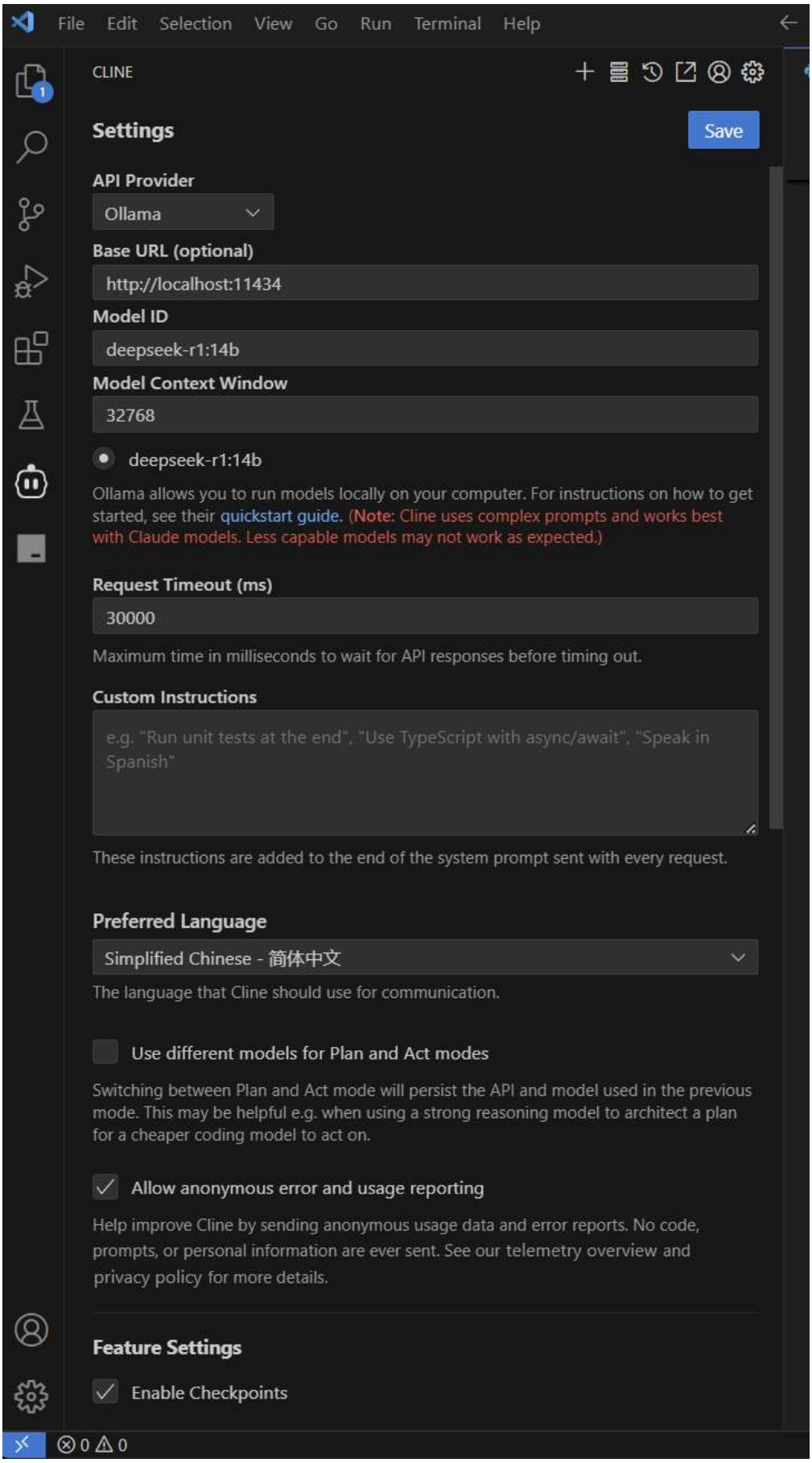
我们也可以本地部署大模型，本地部署的好处是省去了 token 费用，不需要支付调用大模型的年费，同时所有的数据都在本地，安全性和保密性更好。我的本地电脑硬件是这样的：










File Edit Selection View Go Run Terminal Help
CLINE
Settings
Save
API Provider
Ollama
Base URL (optional)
http://localhost:11434
Model ID
deepseek-r1:14b
Model Context Window
32768
deepseek-r1:14b
Ollama allows you to run models locally on your computer. For instructions on how to get started, see their quickstart guide. (Note: Cline uses complex prompts and works best with Claude models. Less capable models may not work as expected.)
Request Timeout (ms)
30000
Maximum time in milliseconds to wait for API responses before timing out.
Custom Instructions
e.g. "Run unit tests at the end", "Use TypeScript with async/await", "Speak in Spanish"
These instructions are added to the end of the system prompt sent with every request.
Preferred Language
Simplified Chinese - 简体中文
The language that Cline should use for communication.
Use different models for Plan and Act modes
Switching between Plan and Act mode will persist the API and model used in the previous mode. This may be helpful e.g. when using a strong reasoning model to architect a plan for a cheaper coding model to act on.
Allow anonymous error and usage reporting
Help improve Cline by sending anonymous usage data and error reports. No code, prompts, or personal information are ever sent. See our telemetry overview and privacy policy for more details.
Feature Settings
Enable Checkpoints
0 0

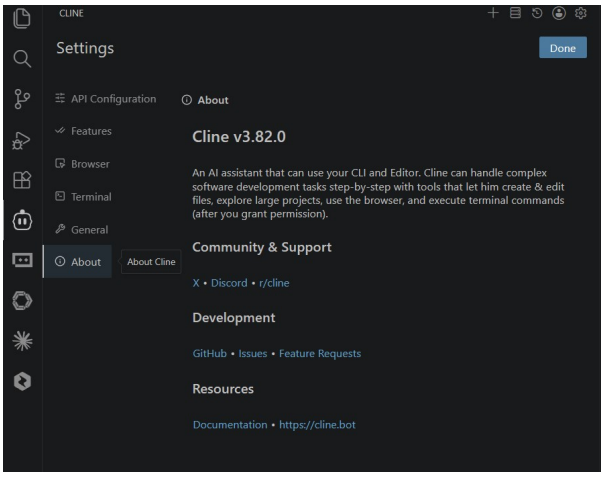
Ollama 运行 google/gemma3:31b 大模型



图片的底部显示的大模型是 ollama:gemma4:31b-cloud

在最新的 Cline v3.82.0 中，有时候经常会有这个提示而中断任务，比较讨厌：

Cline uses complex prompts and iterative task execution that may be challenging for less capable models. For best results, it's recommended to use Claude 4.5 Sonnet for its advanced agentic coding capabilities.



为了解决这个问题，减少它的错误，需要做一下配置优化：

1， 关闭 Native Tool Call

在 Features 设置里把 Native Tool Call 关掉，强制使用通用 XML 格式调用工具，大幅降低格式错误概率。

2， 开启 Strict Plan Mode

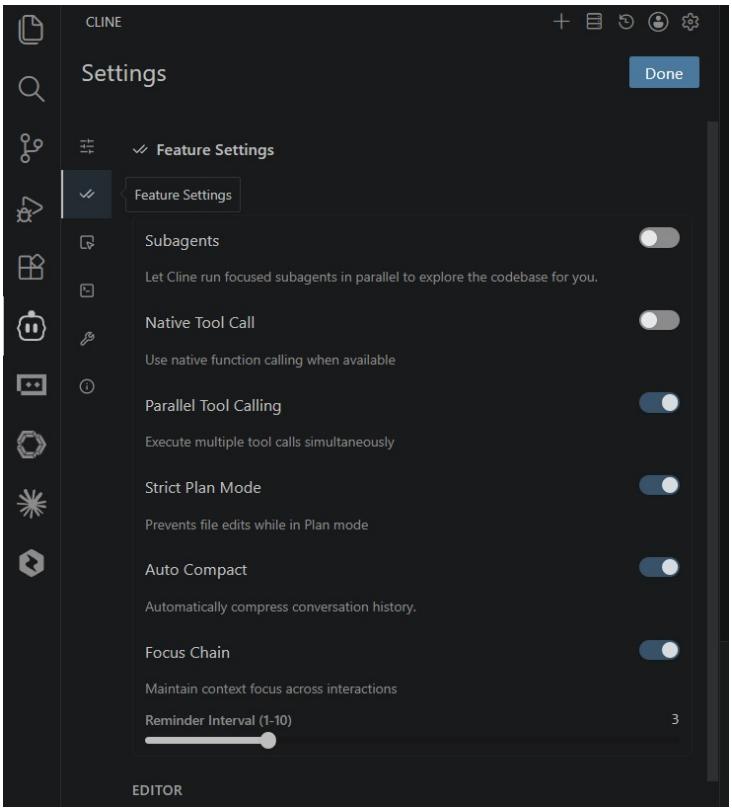
强制模型先写计划再执行，避免乱输出工具调用，减少格式错误。

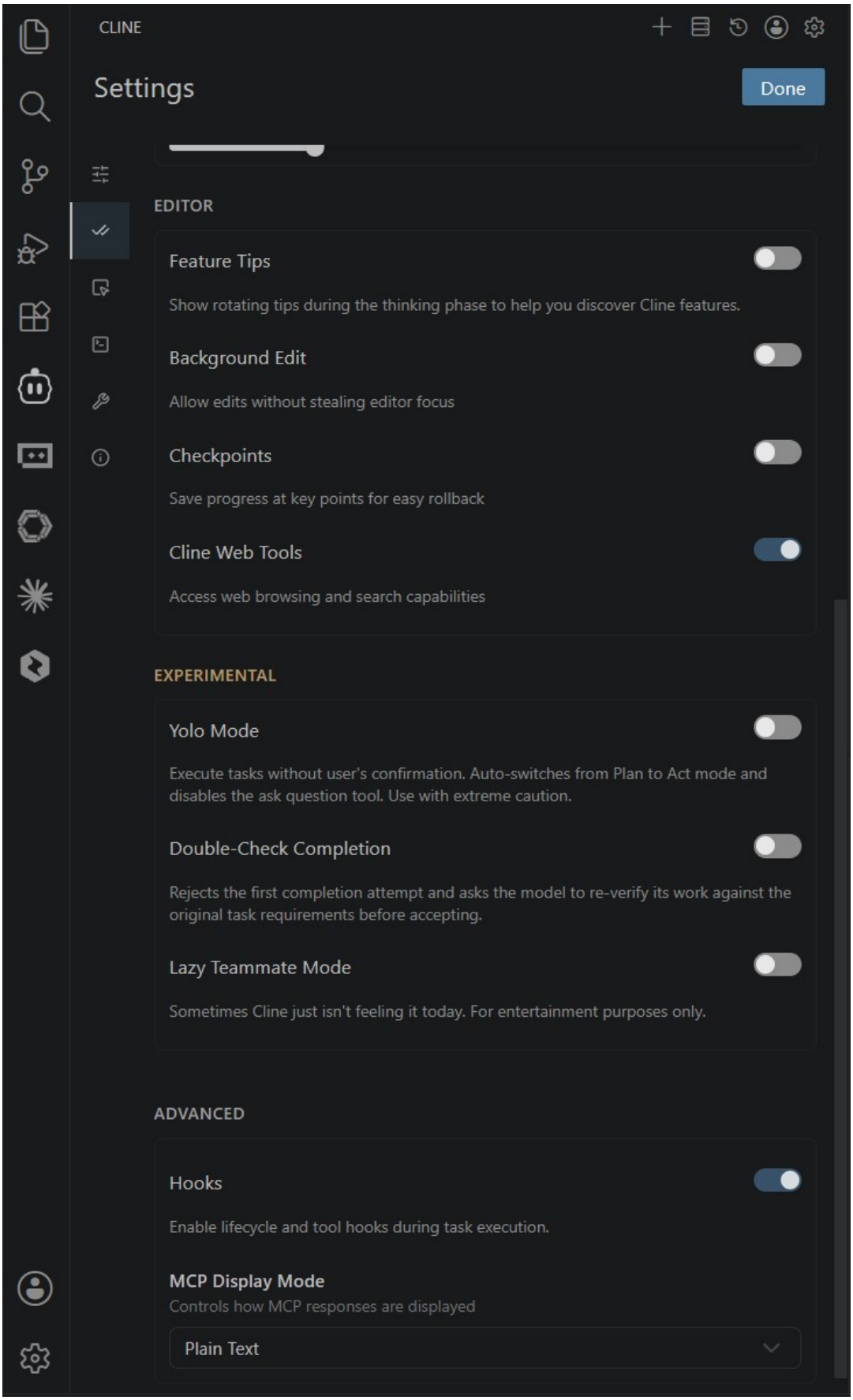
3， 把 Reminder Interval 调低

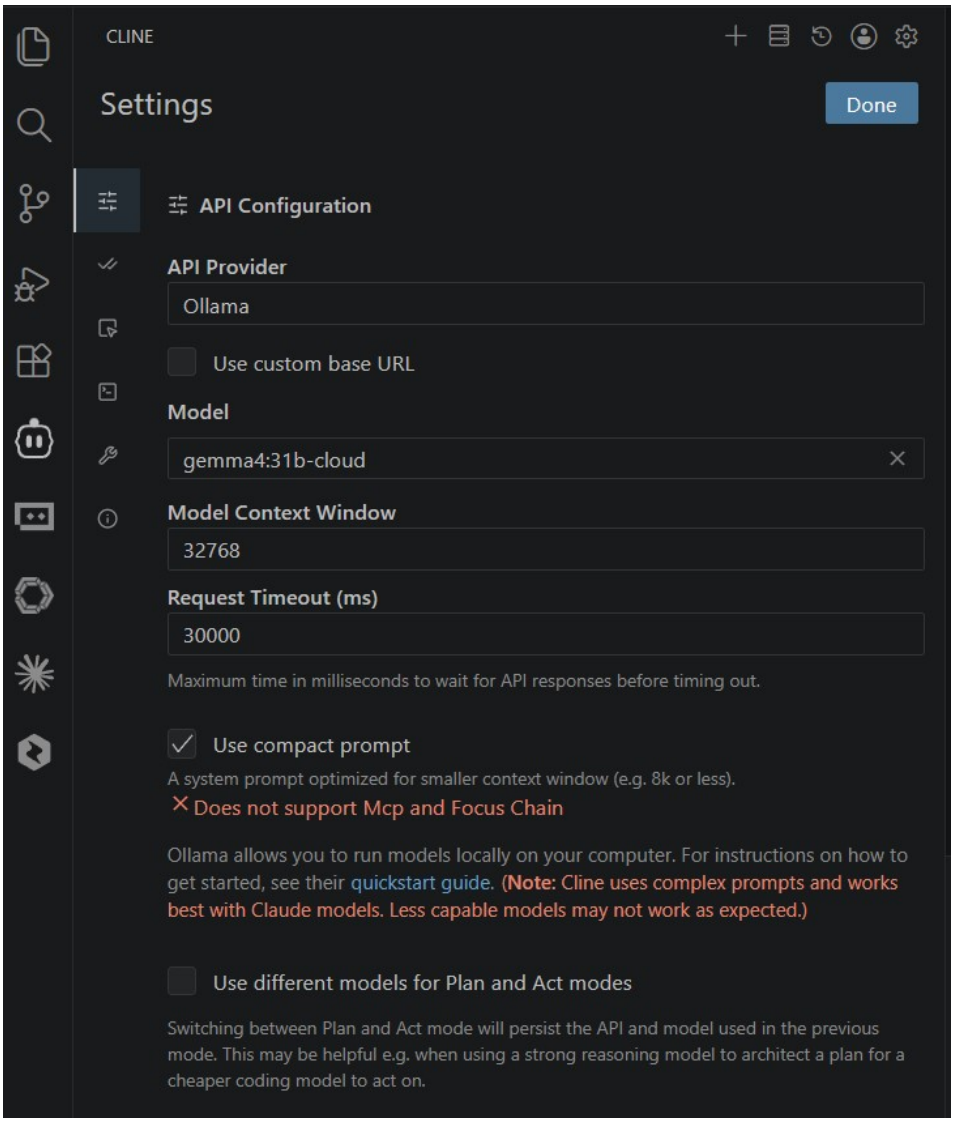
把滑块调到 1-3，减少模型 “忘记规则” 的概率

4， 开启 Use compact prompt

简化提示词，降低模型理解压力。

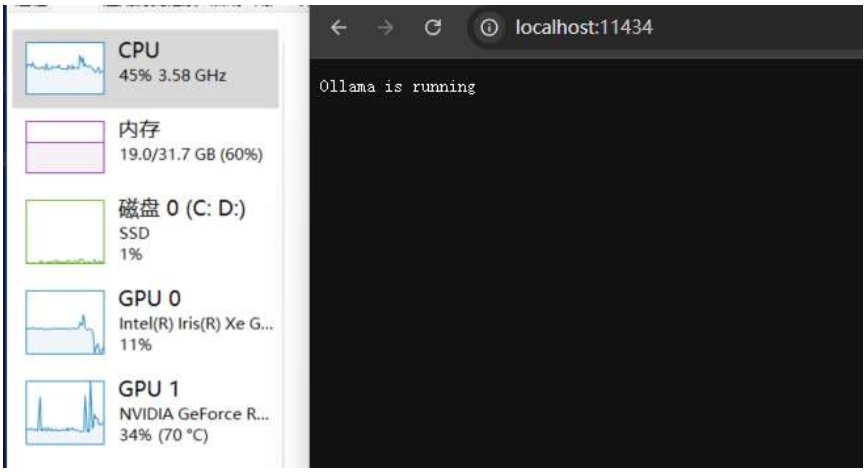



CLINE
Settings
Done
EDITOR
Feature Tips
Show rotating tips during the thinking phase to help you discover Cline features.
Background Edit
Allow edits without stealing editor focus
Checkpoints
Save progress at key points for easy rollback
Cline Web Tools
Access web browsing and search capabilities
EXPERIMENTAL
Yolo Mode
Execute tasks without user's confirmation. Auto-switches from Plan to Act mode and disables the ask question tool. Use with extreme caution.
Double-Check Completion
Rejects the first completion attempt and asks the model to re-verify its work against the original task requirements before accepting.
Lazy Teammate Mode
Sometimes Cline just isn't feeling it today. For entertainment purposes only.
ADVANCED
Hooks
Enable lifecycle and tool hooks during task execution.
MCP Display Mode
Controls how MCP responses are displayed
Plain Text

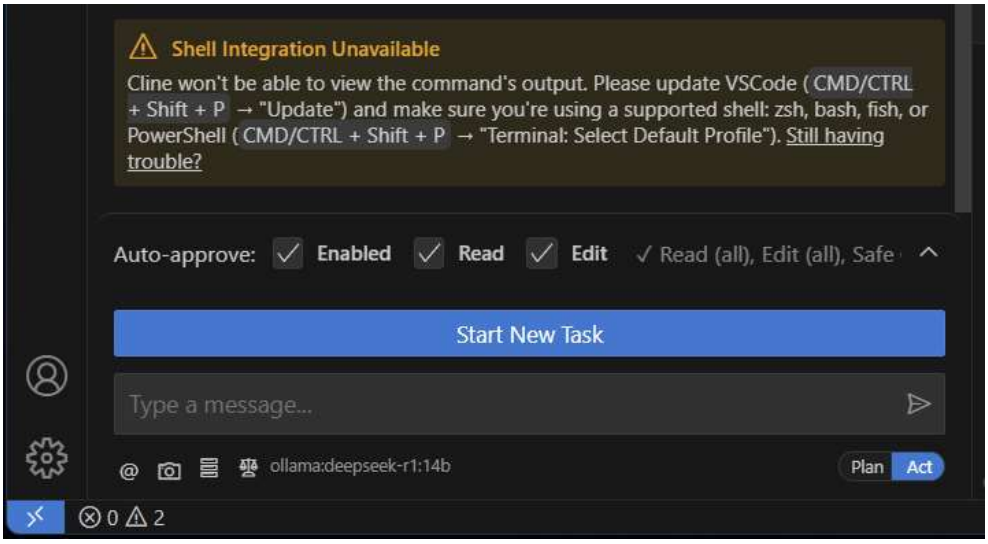




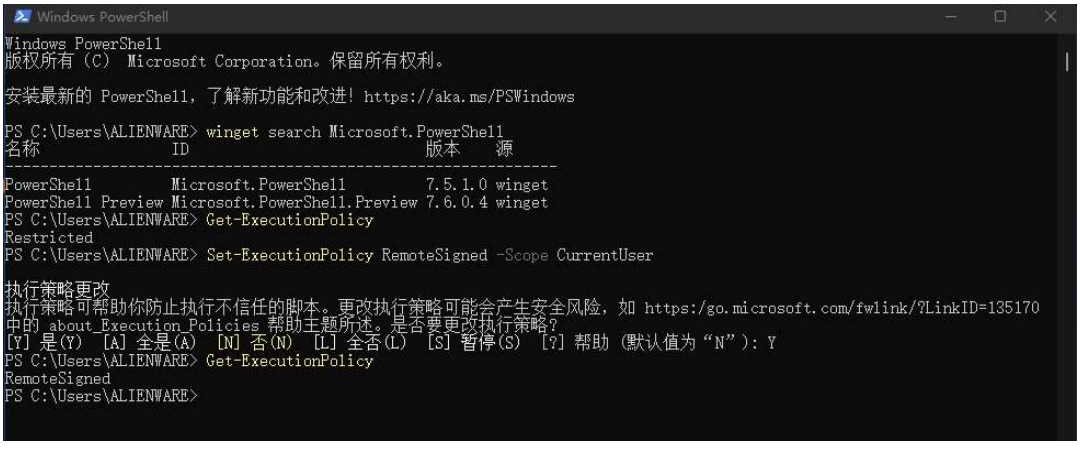
Cline 所有的配置都在上面，配置完成，这个提示没有了，不会中途中断任务。



如果你在使用 Cline 的时候有这个提示：



这是因为 Windows PowerShell 的安全策略设置导致的，按照下面的设置改一下就可以了。



Get-ExecutionPolicy

对于 VS Code 中的开发工作，一般建议使用 RemoteSigned 策略。它允许本地创建的脚本不受限制地运行，同时维护下载脚本的安全性。

Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

此时策略将会调整为 RemoteSigned

5 LM Studio 本地部署 Gemma4 大模型

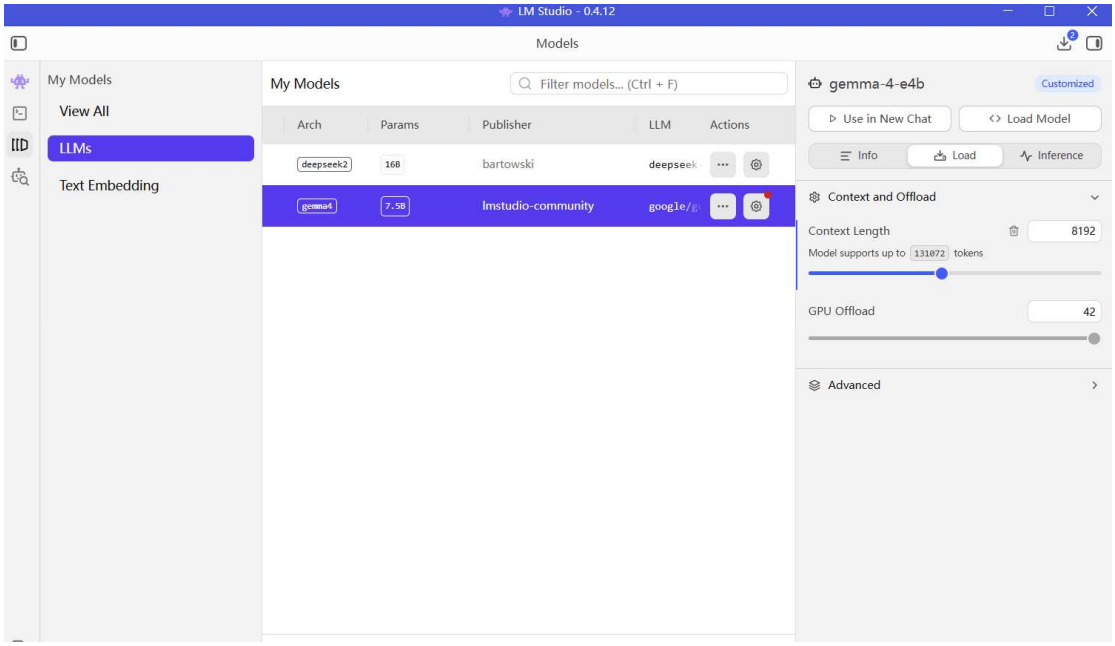
因为 Ollama 订阅政策的变化，算力收费，免费用户 API 调度变慢，已经不适合程序员了。在此我推荐 LM Studio 部署 Gemma4 和 Qwen3.6, 测试下来速度还可以的，非常好用。

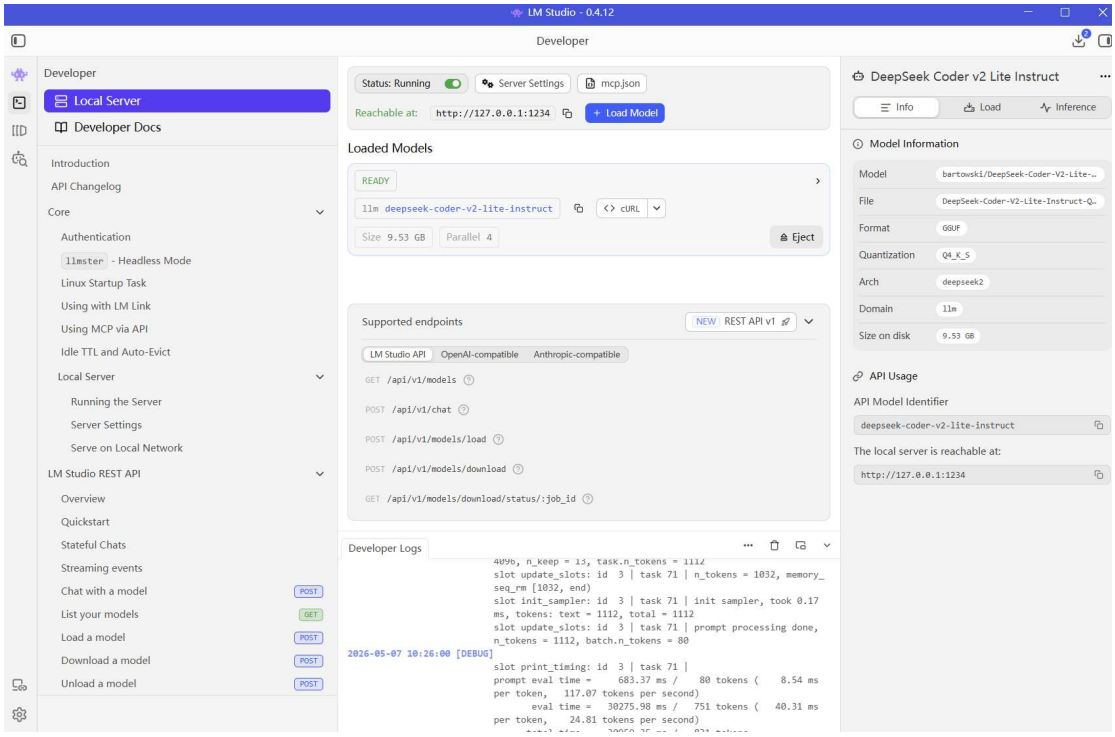
搜查 Continue 这个插件， 安装这个插件，这个插件会 autodect 本地大模型。



可以看到我的 Continue 工具显示是 google/gemma-4-e4b 的大模型。

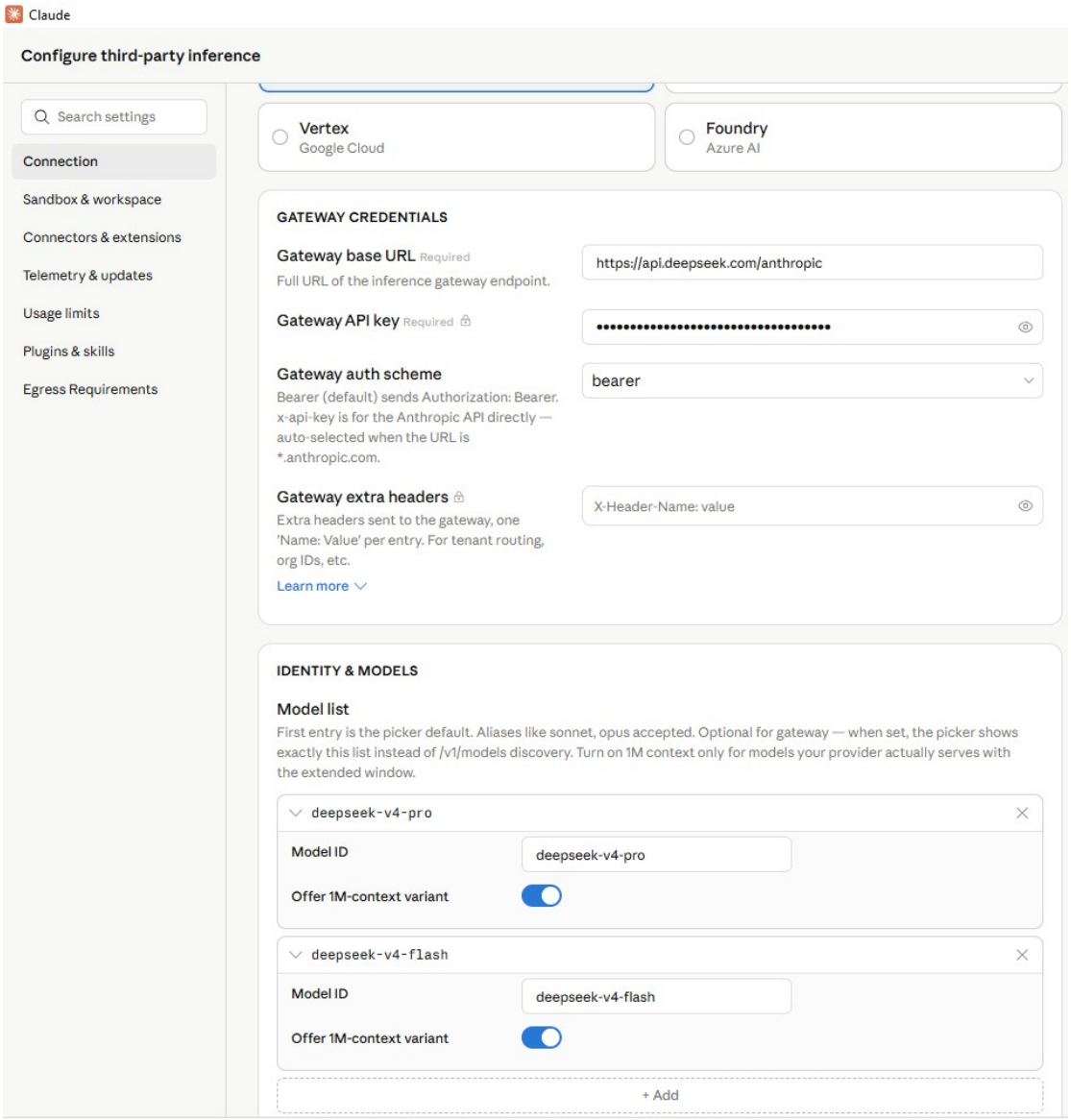
安装 LM Studio，下载能适配你的电脑配置的大模型。
LM Studio 比 Ollama 的配置更灵活， 可以 Partial GPU Offload Possible

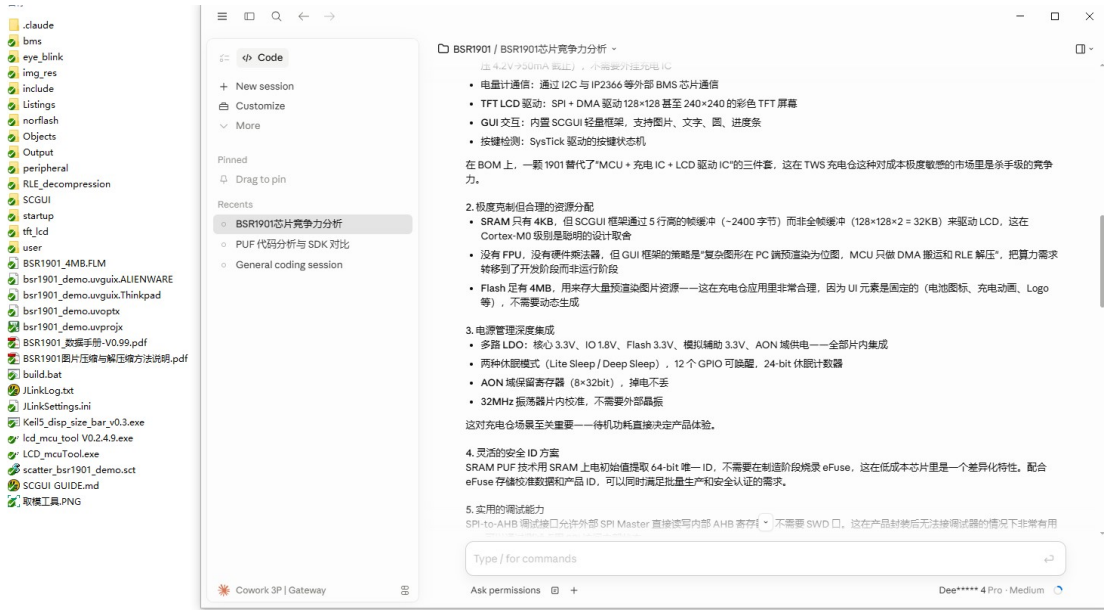




6 Claude + DeepSeek-V4 大模型

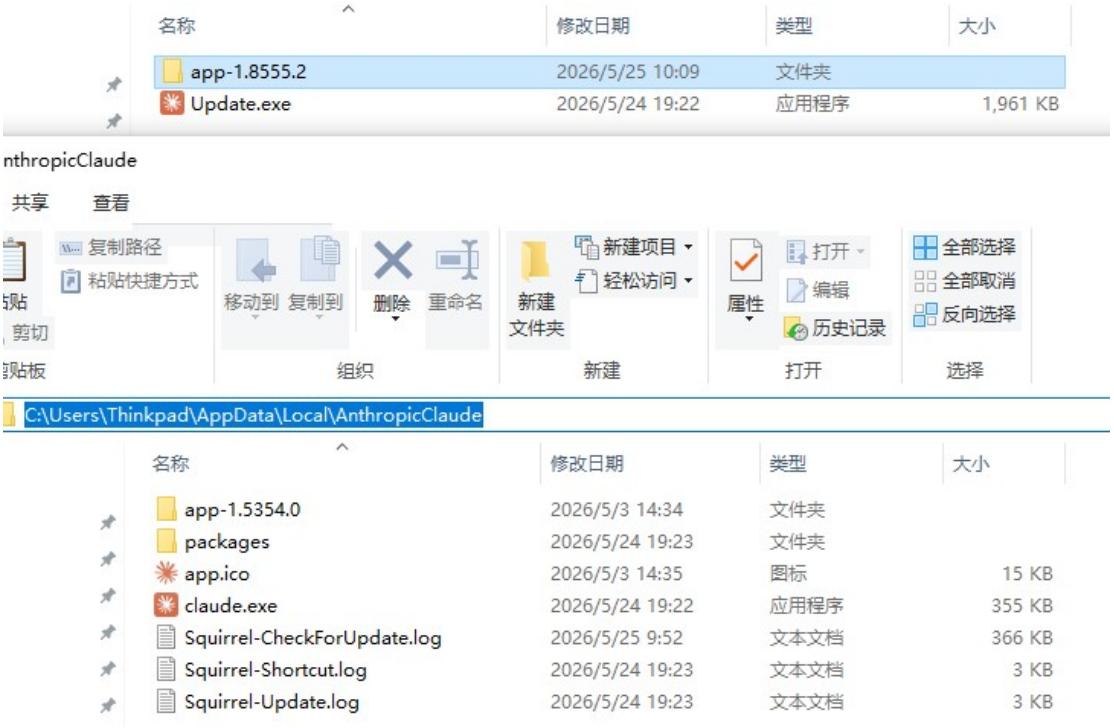
Claude + DeepSeek-V4





Claude 从 1.6259.1 版本开始，Anthropic 强制要求网关配置的 inferenceModels 必须是官方 Claude 模型（如 claude-sonnet-4-5），不接受 DeepSeek 这类第三方模型 ID。一旦更新了 Claude Desktop 客户端，所有的配置都会失效。

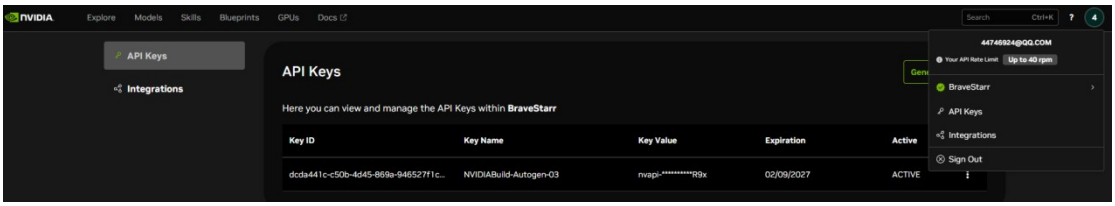
解决办法是：

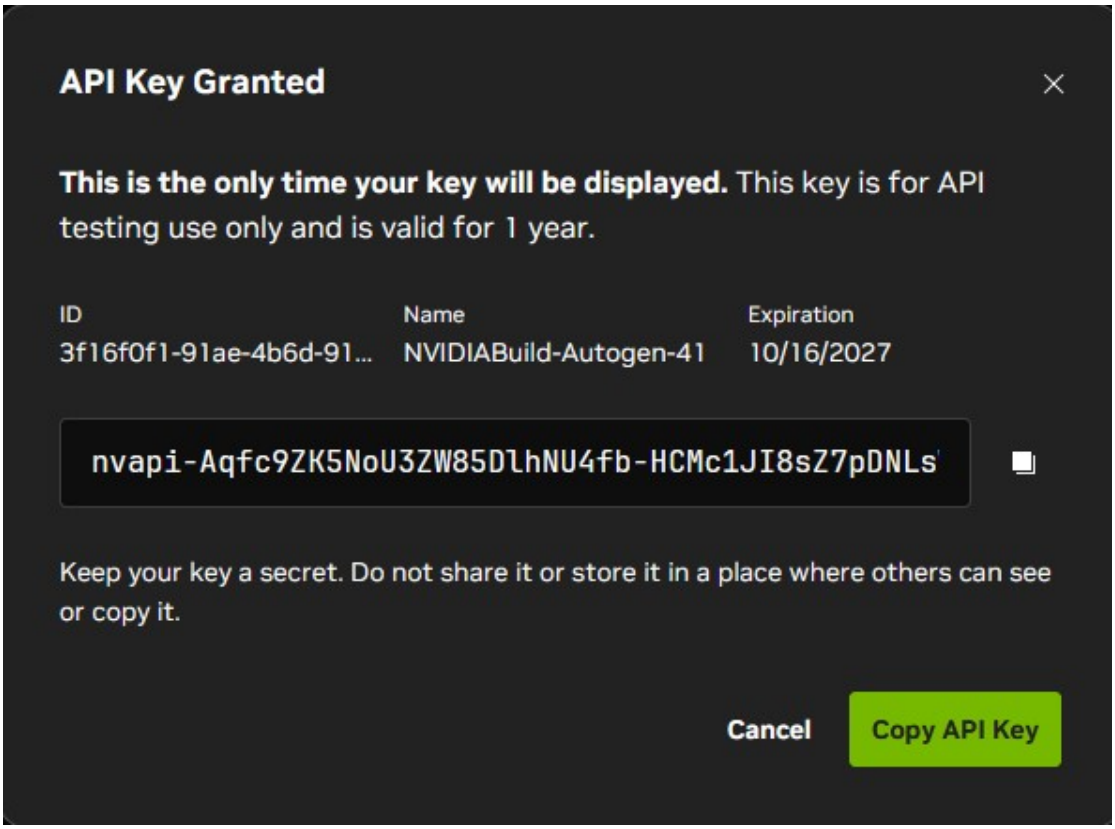


按照上面的操作，立刻恢复成可以正常使用的 DeepSeek 第三方模型。

7 Cline+ NVIDIA 大模型

免费大模型，登录网站 https://build.nvidia.com/ 注册：





配置模型：

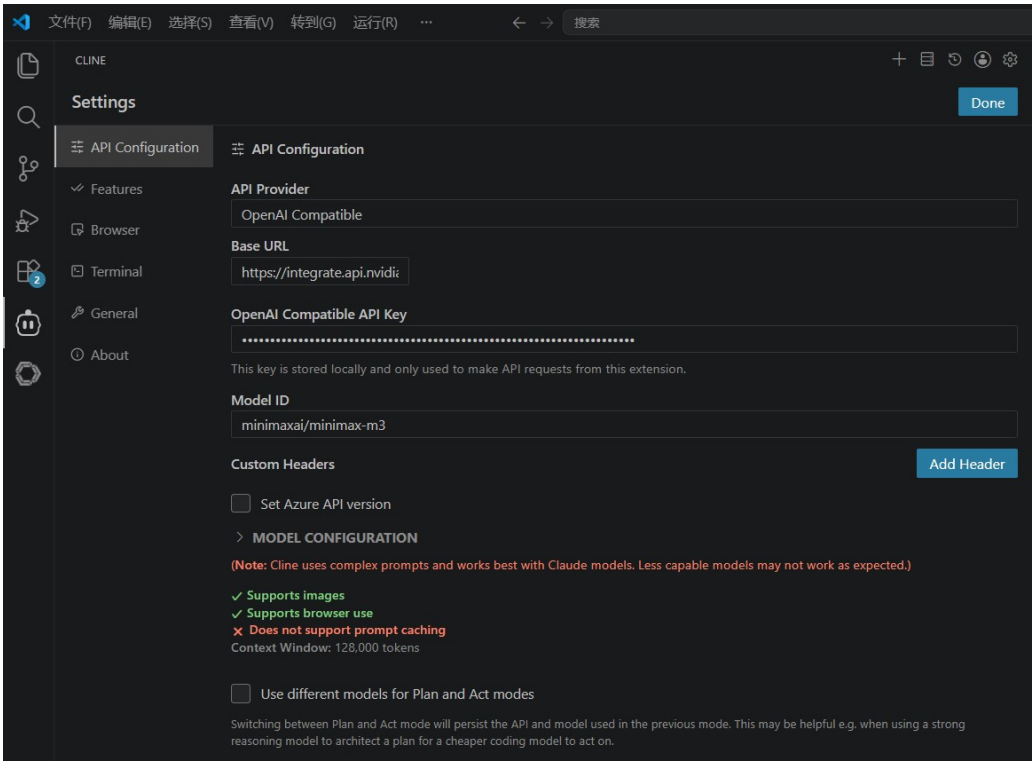


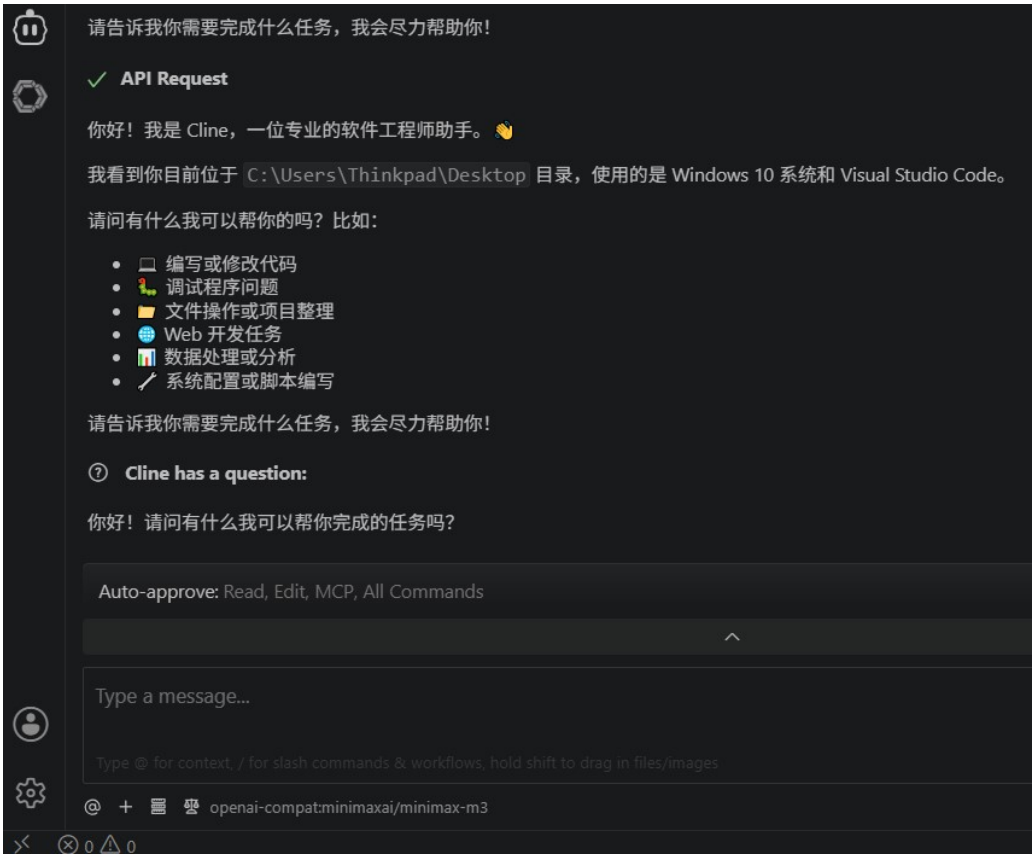
我的 Cline + VS Code 是这样配置的：

Base URL

https://integrate.api.nvidia.com/v1

选择模型 minimaxai/minimax-m3 或者 google/gemma-4-31b-it





8 打印 log 信息

通过硬件 UART 打印 log，是最通用、方便的调试方法。

打印 log 信息的方法：

```
adc_voltage_ch5=Get_Vbat_Voltage();
printf("\r\nGet_Vbat_Voltage adc_voltage_ch5 = %d",adc_voltage_ch5);
//printf("\r\nnGet_Vbat_Voltage adc_voltage_ch5 = 0x%x",adc_voltage_ch5);
//printf("gek1109_sdk\r\n");
```

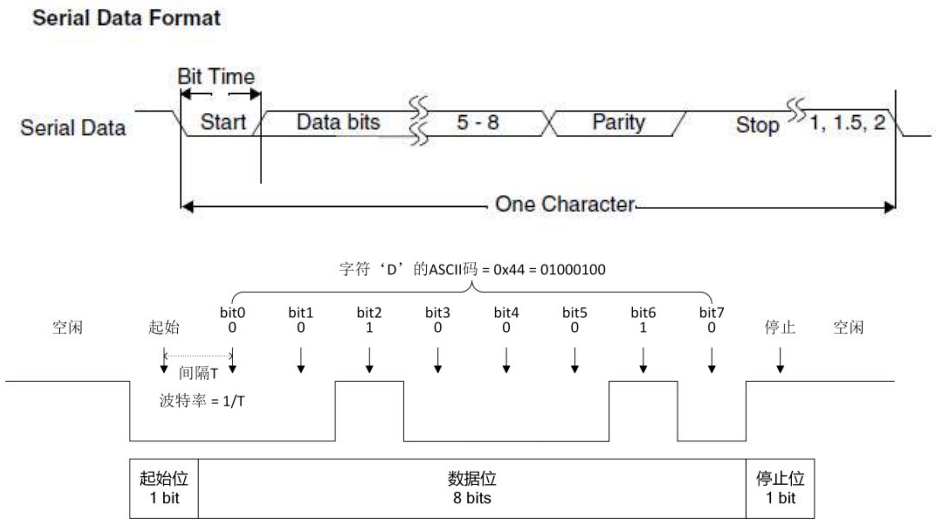
在调用之前，需要在该文件的头上，包含一个这样的文件：

```
#include <stdio.h>
```

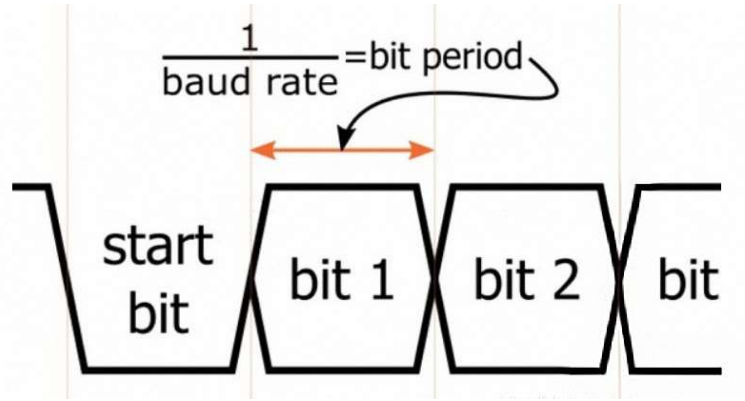
同时需要在工程代码中，重新定义这个函数：

```
/* retarget the C library printf function to the USART */
int fputc(int ch, FILE *f)
{
      Gecko_Uart0_Send_Char(ch);
      return ch;
}
```

UART 的数据格式：



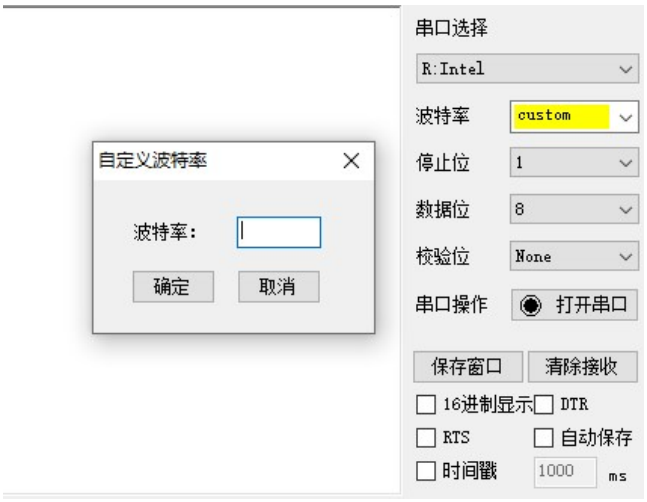
常见的有 9600bps、115200bps 等，关于波特率这里说明一下：



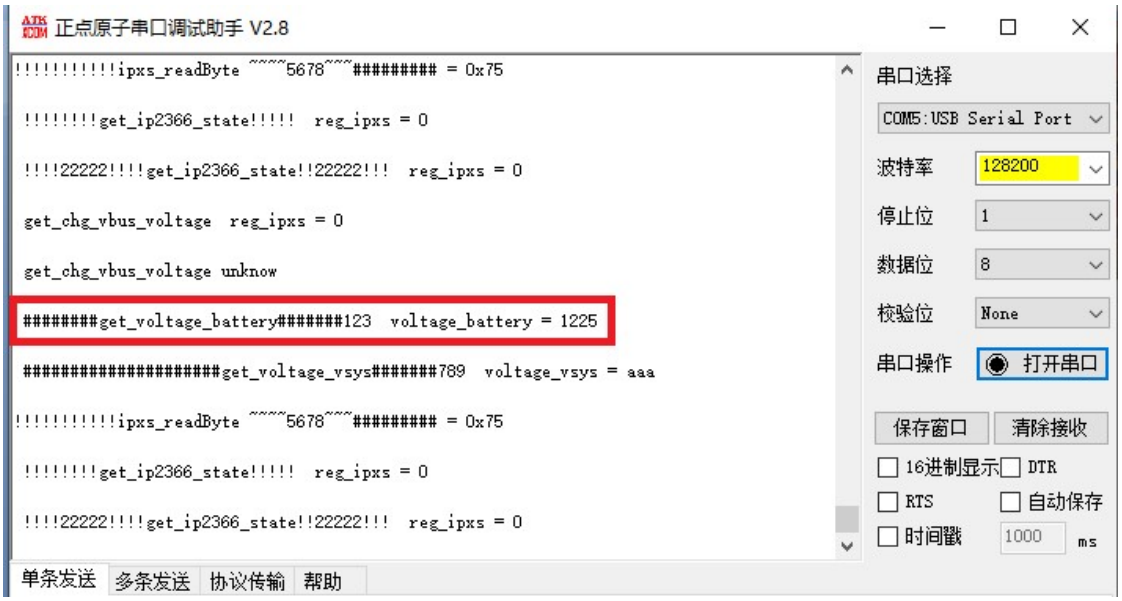
在只有起始位和 1 位停止位的情况下，传输一个字节的数据包长度为 10 个 bits ，这种情况下的每秒钟可以传输的字节数为 ：9600/10=**960** bytes 。从下图中，我们可以看到在 115200bps 时，一个 bit 的宽度是 8.68us，供客户调试参考。

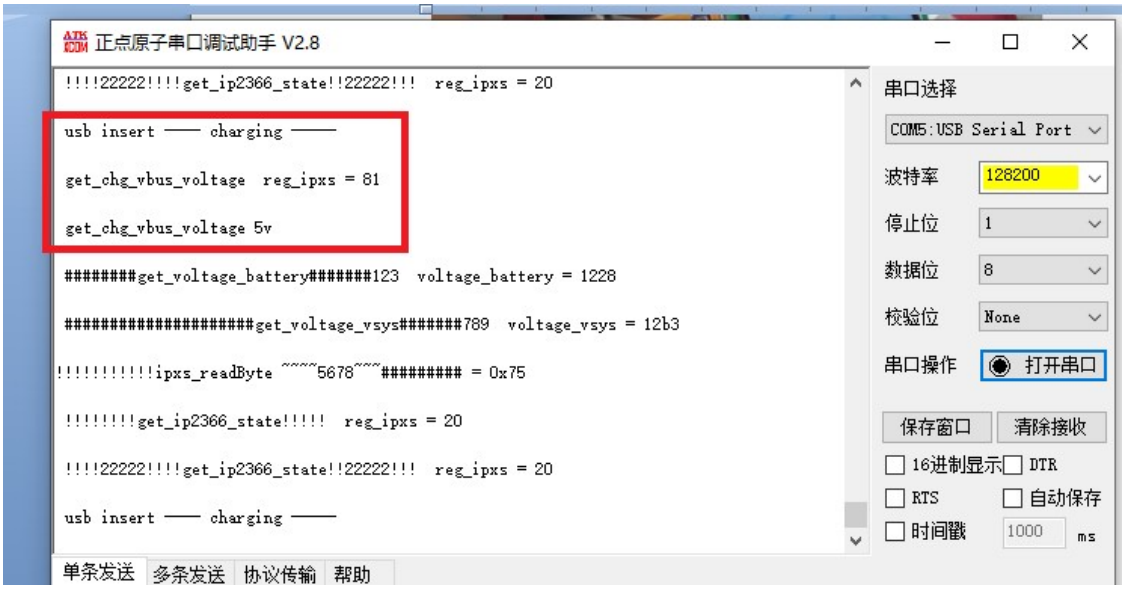
Transmission speed				Real transmission speed	
Bauds	**Bit/s**	**Bit duration**	**Speed**	**Speed**	**Byte duration**
50 Bd	50 bits/s	20.000 ms	6.25 bytes/s	5 bytes/s	200.000 ms
75 Bd	75 bits/s	13.333 ms	9.375 bytes/s	7.5 bytes/s	133.333 ms
110 Bd	110 bits/s	9.091 ms	13.75 bytes/s	11 bytes/s	90.909 ms
134 Bd	134 bits/s	7.463 ms	16.75 bytes/s	13.4 bytes/s	74.627 ms
150 Bd	150 bits/s	6.667 ms	18.75 bytes/s	15 bytes/s	66.667 ms
200 Bd	200 bits/s	5.000 ms	25 bytes/s	20 bytes/s	50.000 ms
300 Bd	300 bits/s	3.333 ms	37.5 bytes/s	30 bytes/s	33.333 ms
600 Bd	600 bits/s	1.667 ms	75 bytes/s	60 bytes/s	16.667 ms
1200 Bd	1200 bits/s	833.333 µs	150 bytes/s	120 bytes/s	8.333 ms
1800 Bd	1800 bits/s	555.556 µs	225 bytes/s	180 bytes/s	5.556 ms
2400 Bd	2400 bits/s	416.667 µs	300 bytes/s	240 bytes/s	4.167 ms
4800 Bd	4800 bits/s	208.333 µs	600 bytes/s	480 bytes/s	2.083 ms
9600 Bd	**9600 bits/s**	**104.167 µs**	**1200 bytes/s**	**960 bytes/s**	**1.042 ms**
19200 Bd	19200 bits/s	52.083 µs	2400 bytes/s	1920 bytes/s	520.833 µs
28800 Bd	28800 bits/s	34.722 µs	3600 bytes/s	2880 bytes/s	347.222 µs
38400 Bd	38400 bits/s	26.042 µs	4800 bytes/s	3840 bytes/s	260.417 µs
57600 Bd	57600 bits/s	17.361 µs	7200 bytes/s	5760 bytes/s	173.611 µs
76800 Bd	76800 bits/s	13.021 µs	9600 bytes/s	7680 bytes/s	130.208 µs
115200 Bd	**115200 bits/s**	**8.681 µs**	**14400 bytes/s**	**11520 bytes/s**	**86.806 µs**
230400 Bd	230400 bits/s	4.340 µs	28800 bytes/s	23040 bytes/s	43.403 µs
460800 Bd	460800 bits/s	2.170 µs	57600 bytes/s	46080 bytes/s	21.701 µs
576000 Bd	576000 bits/s	1.736 µs	72000 bytes/s	57600 bytes/s	17.361 µs
921600 Bd	921600 bits/s	1.085 µs	115200 bytes/s	92160 bytes/s	10.851 µs

客户在调试的时候，如果因为 1109 的 RC 不准，又不想去手动校准，我们建议可以在网上搜寻一个可以支持任意波特率的串口助手，也就是可以用户自定义波特率的串口助手软件工具， 比如：



这样就免去了手动去校准 1109 RC 时钟精度的复杂流程。可以帮助用户在调试时，比较轻松的打印 log，辅助我们的软件调试。推荐 SuperCom，SerialPortHelper 等，这些都是开源的工具。

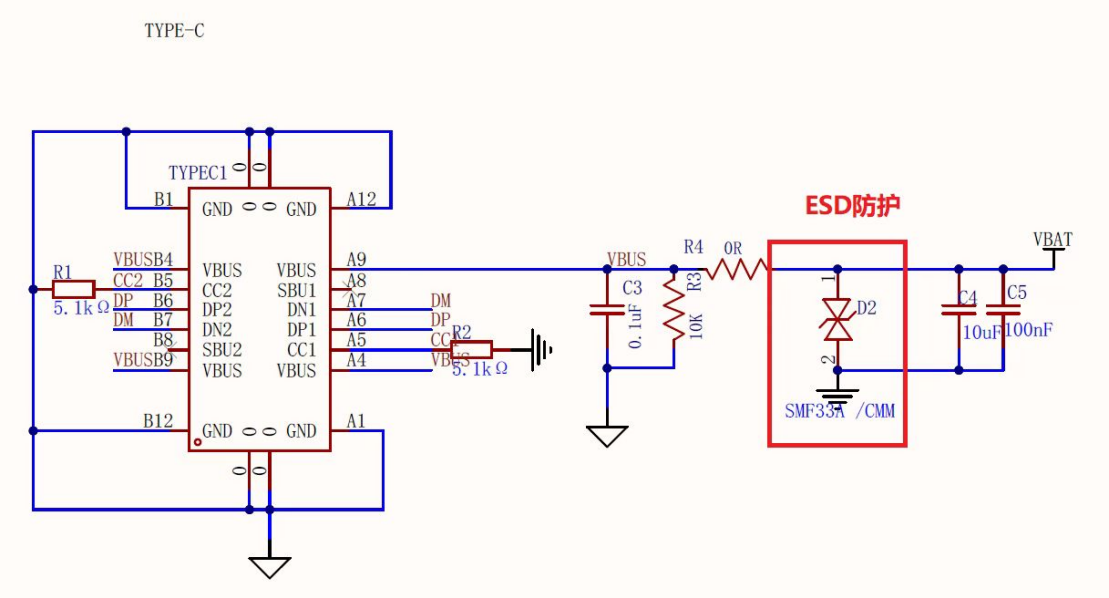




9 在线升级

客户设计的终端产品，通常要求在量产之后，还能继续升级，对于 ARM cpu,我们给出的参考建议是 type-c 接口的 SWD 升级，好处是可以不拆机升级，而且不需要依赖额外的工具，使用 JFlash 就可以更新代码，同时它更新代码速度快，5MHZ 或者 10MHZ 都没有问题。

正常的 type-c 连接图是这样的：



如果希望设备能通过 Type-C 接口供电，需要在 CC1 和 CC2 引脚上各连接一个 5.1kΩ 下拉电阻到地。这样主机才能识别并输出 5V 电压。

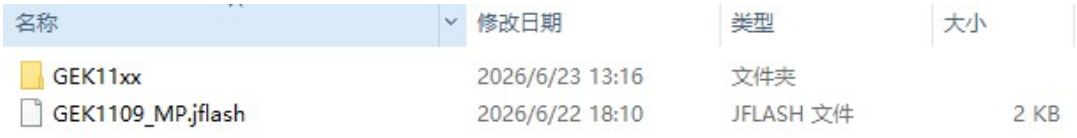
建议连接：

DP ------SWCLK（single wire clock）
DN------SWDIO（single wire data）
SBU-----RSTN（reset），可以不连接

SBU 连接 RSTN 是为了配合下载工具，在任何情况下（比如 swd 管脚复用为其它功能），都能 reset cpu，让芯片进入正常的下载。为确保信号完整性，建议在 DP、DN 和 SBU 线路上预留 ESD 保护器件的位置。

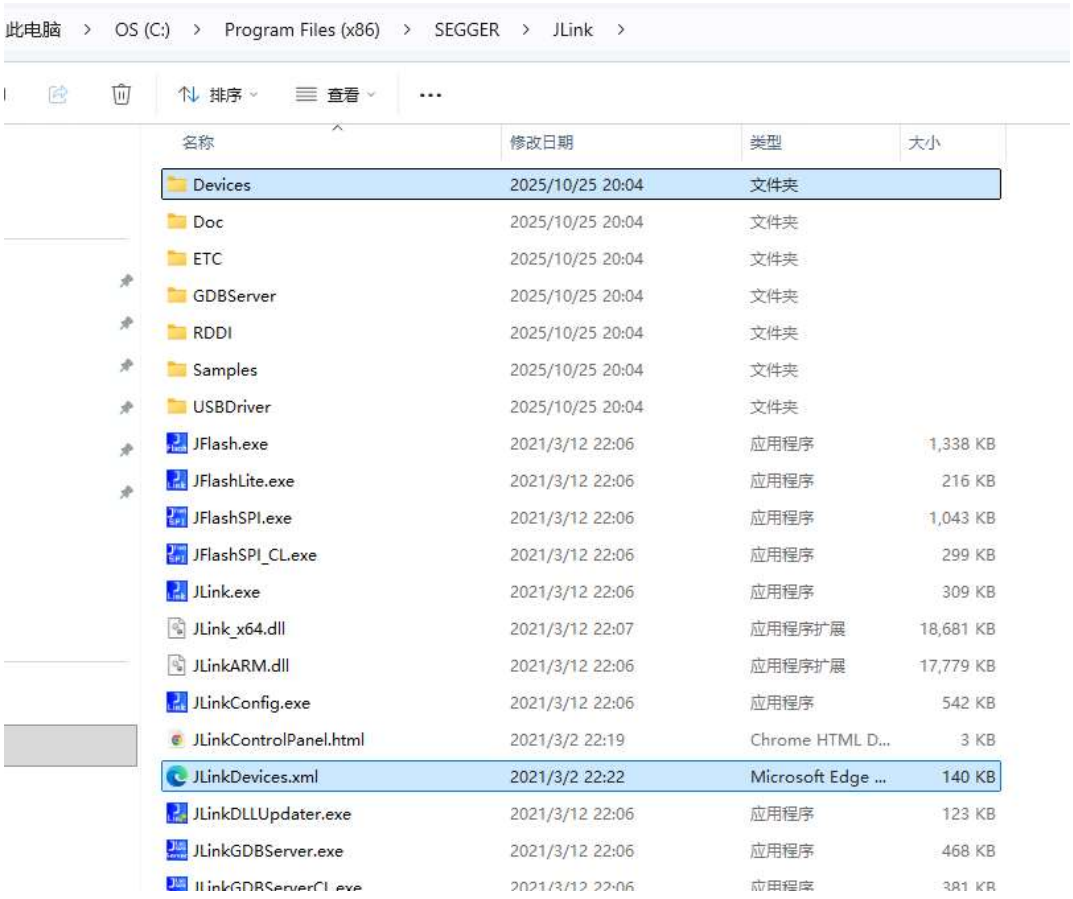
10 量产 JFlash 烧录

我们在电脑上，安装了 J-Flash（我安装的是 J-Flash V6.98b）工具之后，需要安装一个 1109 或者 1901 的补丁。

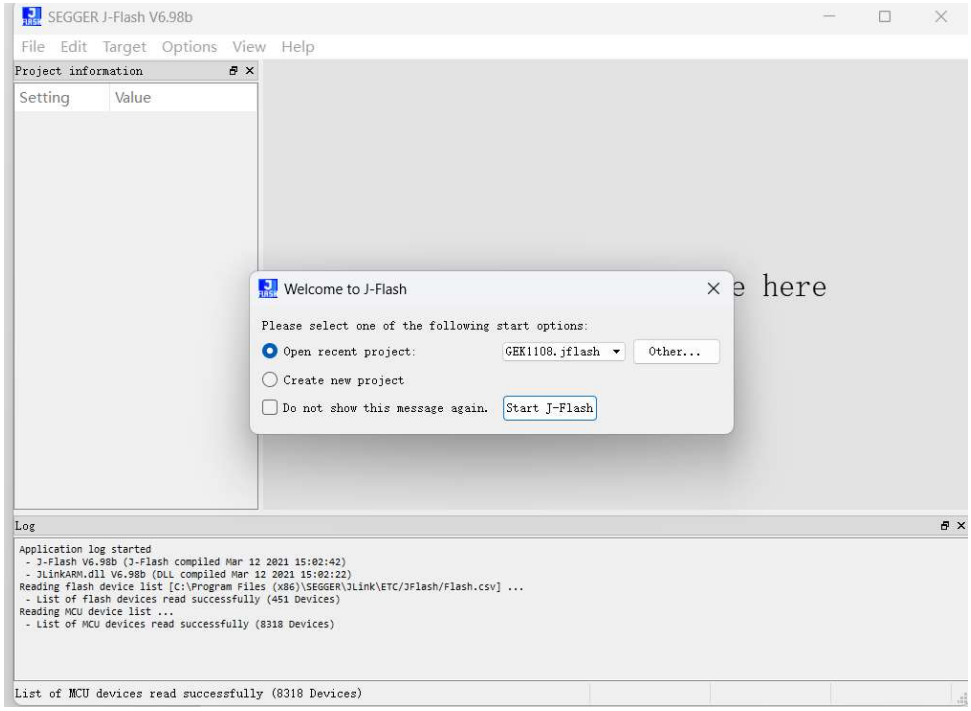


复制补丁到 Devices 目录，同时在 JLinkDevices.xml 里面增加一条设备信息：

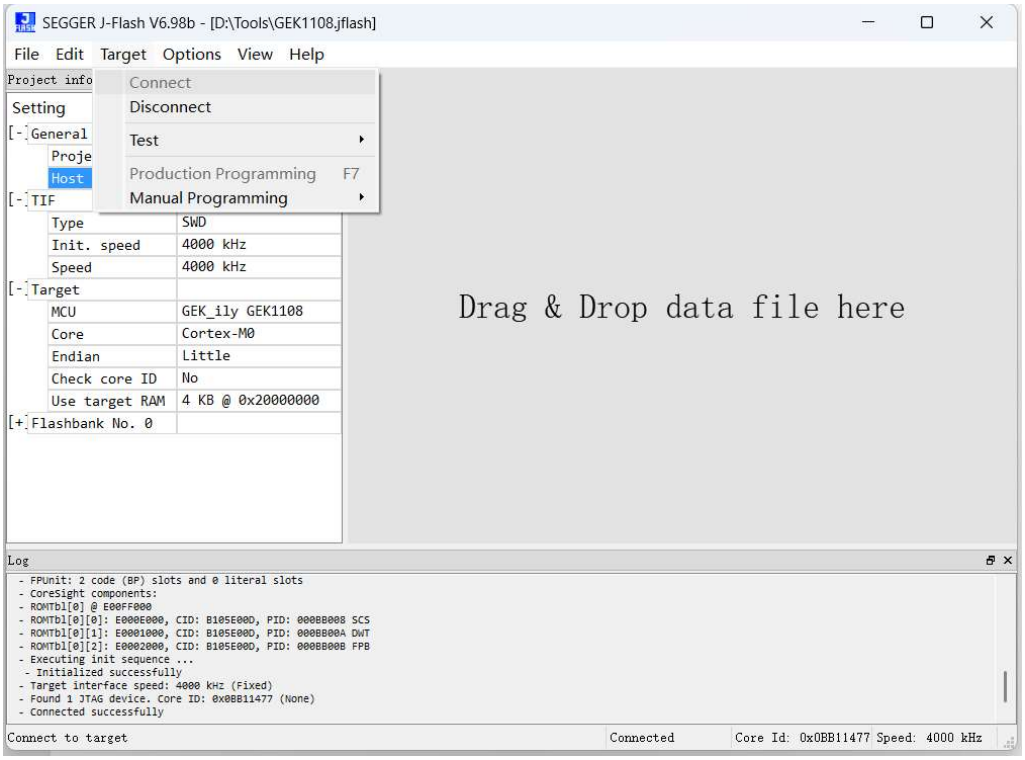




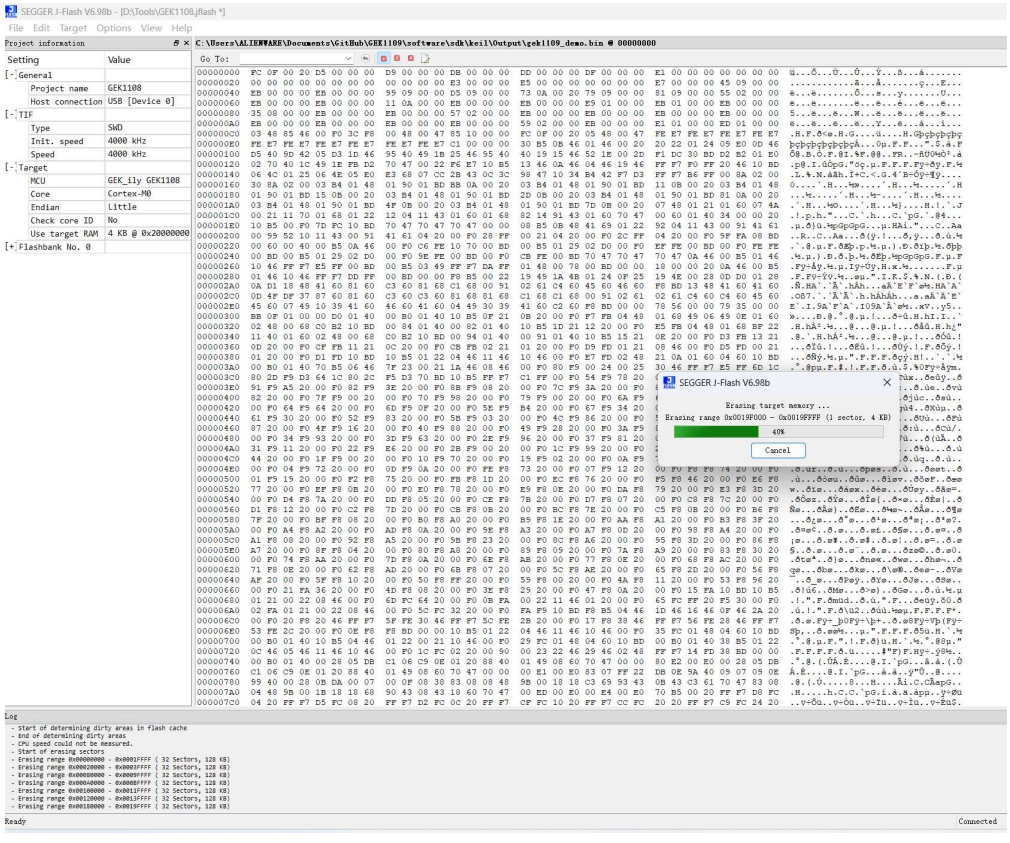
找到 J-Flash 的安装位置，并用我们提供的补丁，直接覆盖掉上面的 Devices 文件夹和 xml 文件。再次打开 J-Flash 的时候，就可以看到这个界面：



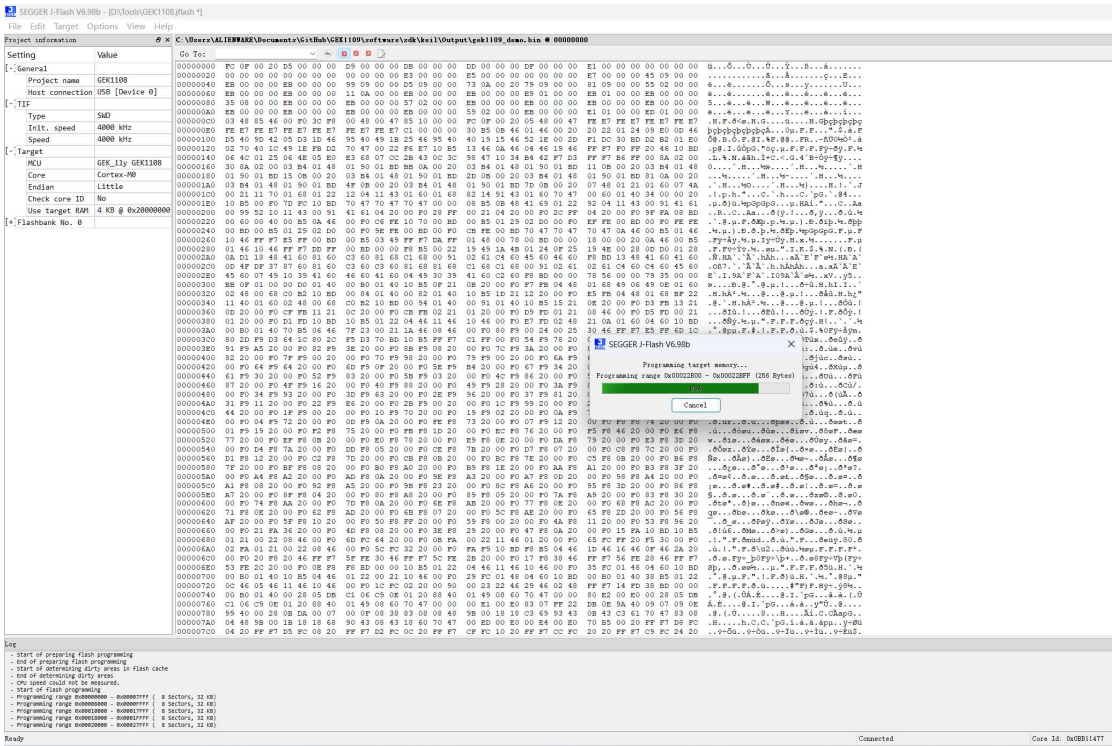
在 Target 菜单下，Connect 就可以链接上我们的 1109/1901 芯片。Log 信息也显示 Jlink 已经连接正常了：



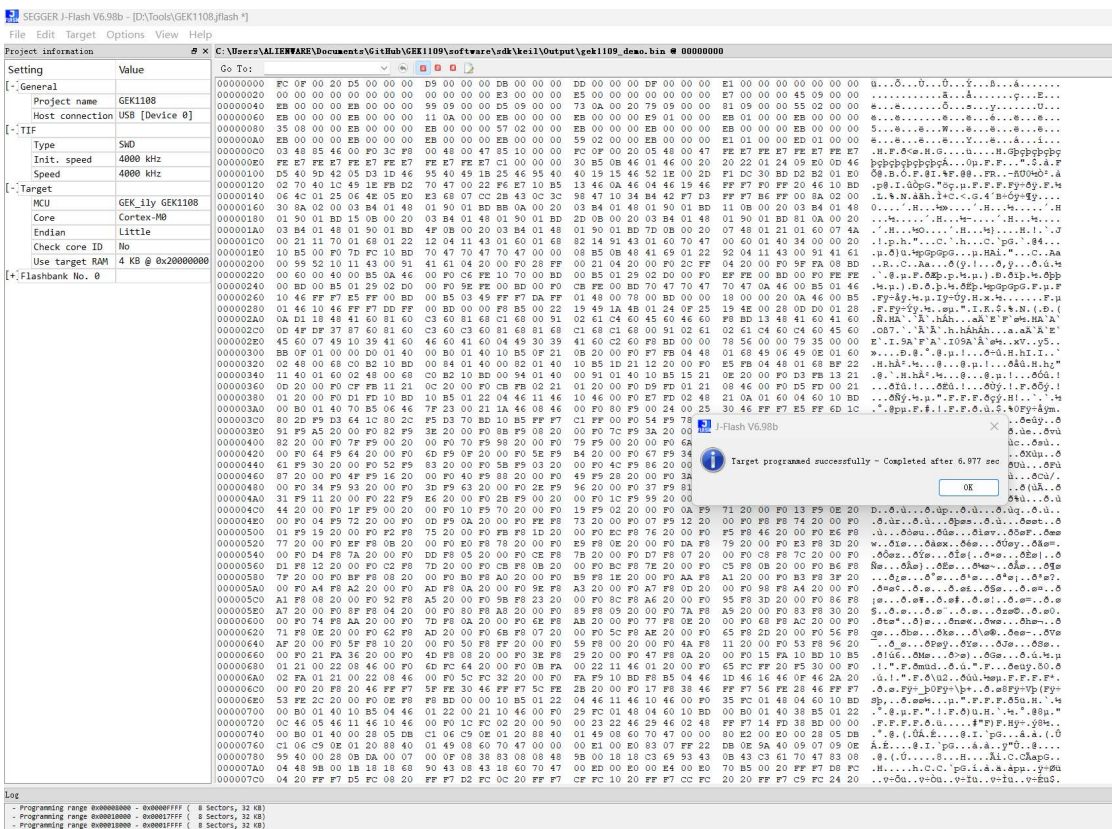
这个时候，我们可以进入 **Manual Programming** 菜单，**Erase Sectors** 擦除掉芯片 **Flash** 的程序。



正在 **program Flash**


SEGGER J-Flash V6.98b - [D:\Tools\GEK1108.jflash *]
File Edit Target Options View Help
Project information
Setting Value
[-] General
Project name GEK1108
Host connection USB [Device 0]
[-] TIF
Type SwD
Init. speed 4000 kHz
Speed 4000 kHz
[-] Target
MCU GEK_ily GEK1108
Core Cortex-M0
Endian Little
Check core ID No
Use target RAM 4 KB @ 0x20000000
[+] Flashbank No. 0
C:\Users\ALIENWARE\Documents\GitHub\GEK1109\software\sdk\keil\Output\gek1109_demo.bin @ 00000000
Go To:
SEGGER J-Flash V6.98b
Programming target memory...
Programming range 0x00022B00 - 0x00023BFF (256 Bytes)
0%
Cancel
Log
- Start of preparing flash programming
- End of preparing flash programming
- Start of determining dirty areas in flash cache
- End of determining dirty areas
- CPU speed could not be measured.
- Start of flash programming
- Programming range 0x00000000 - 0x00007FFF (8 Sectors, 32 KB)
- Programming range 0x00010000 - 0x00017FFF (8 Sectors, 32 KB)
- Programming range 0x00018000 - 0x0001FFFF (8 Sectors, 32 KB)
- Programming range 0x00020000 - 0x00027FFF (8 Sectors, 32 KB)
- Programming range 0x00022000 - 0x00023FFF (8 Sectors, 32 KB)
Ready
Connected
Core Id: 0x0BB11477

Flash 烧录成功。


SEGGER J-Flash V6.98b - [D:\Tools\GEK1108.jflash *]
File Edit Target Options View Help
Project information
Setting Value
[-] General
Project name GEK1108
Host connection USB [Device 0]
[-] TIF
Type SwD
Init. speed 4000 kHz
Speed 4000 kHz
[-] Target
MCU GEK_ily GEK1108
Core Cortex-M0
Endian Little
Check core ID No
Use target RAM 4 KB @ 0x20000000
[+] Flashbank No. 0
C:\Users\ALIENWARE\Documents\GitHub\GEK1109\software\sdk\keil\Output\gek1109_demo.bin @ 00000000
Go To:
J-Flash V6.98b
Target programmed successfully - Completed after 6.977 sec
OK
Log
- Programming range 0x00000000 - 0x0000FFFF (8 Sectors, 32 KB)
- Programming range 0x00010000 - 0x00017FFF (8 Sectors, 32 KB)
- Programming range 0x00018000 - 0x0001FFFF (8 Sectors, 32 KB)
- Programming range 0x00020000 - 0x00027FFF (8 Sectors, 32 KB)
- Programming range 0x00022000 - 0x00023FFF (1 Sector, 4 KB)
- End of flash programming
- Flash programming performed for 1 range (167936 bytes)
- 0x0000 - 0x28FFF (41 Sectors, 164 KB)
- Start of restoring
- End of restoring
- Target programmed successfully - Completed after 6.977 sec
Ready
Connected

11 修订记录

REV	DATE	AUTHER	CHANGE DESCRIPTION
V1	2024-1-16	Junfeng Zhou	First draft initial Release